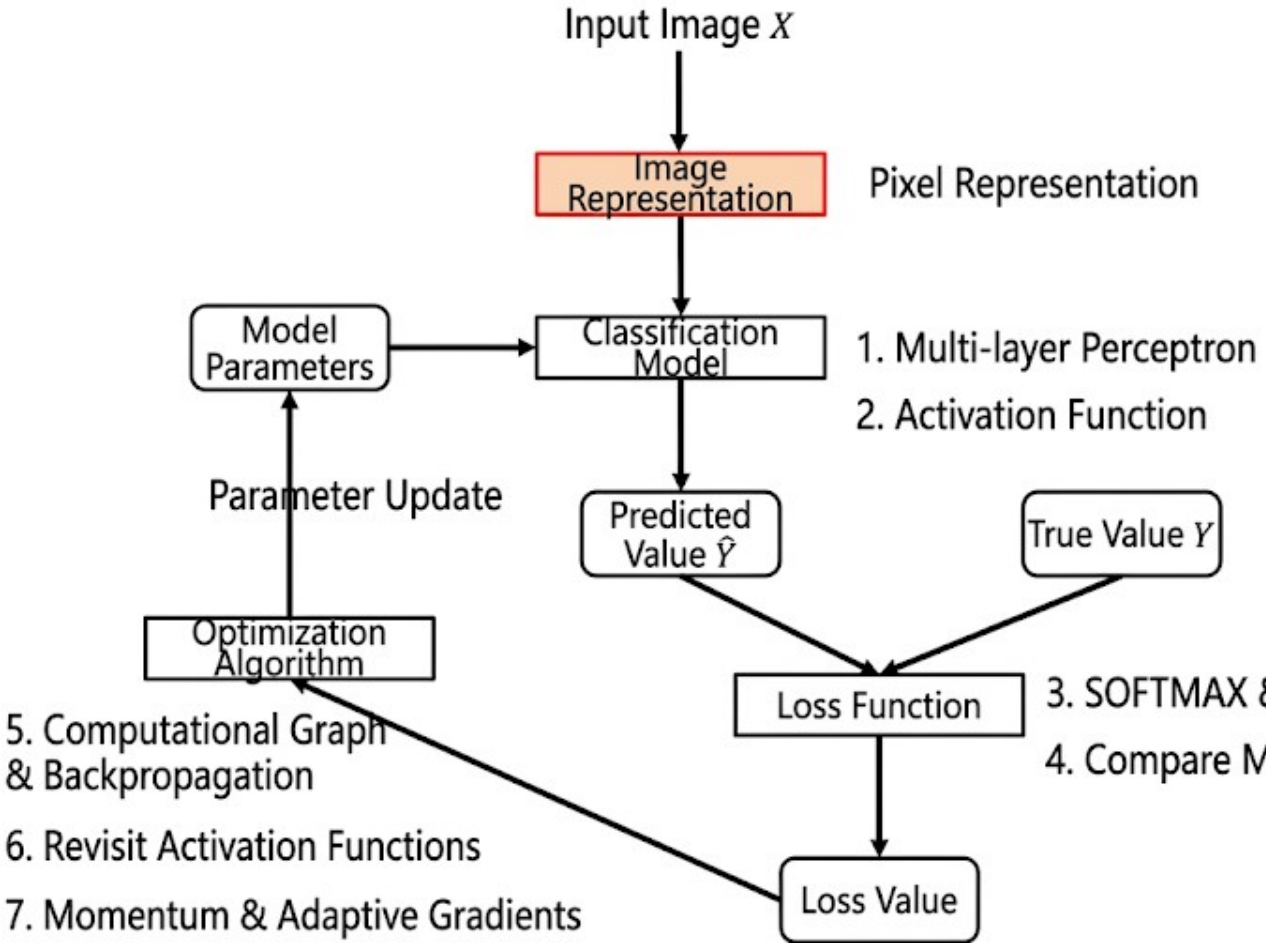
Fully Connected Neural Network

Chen Peng
School of Artificial Intelligence and Data Science
Hebei University of Technology
May 17, 2026

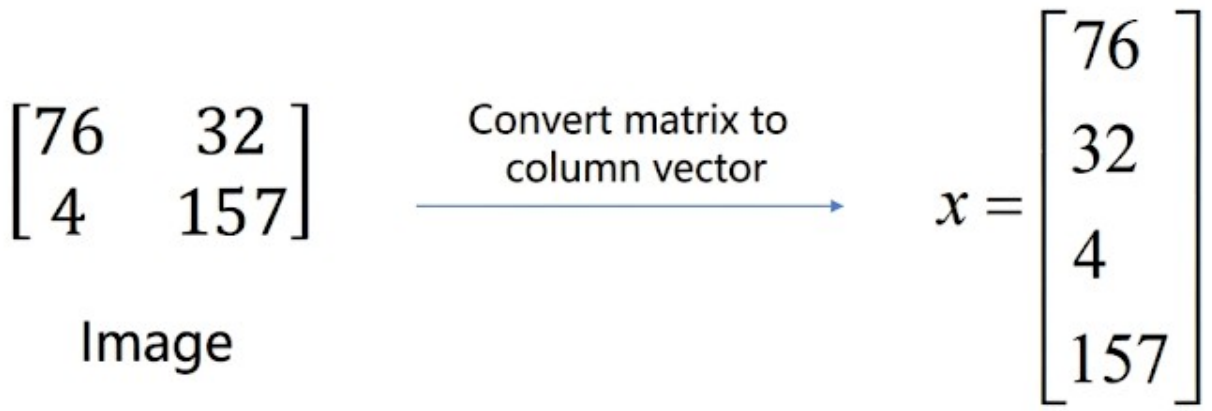


Training Process

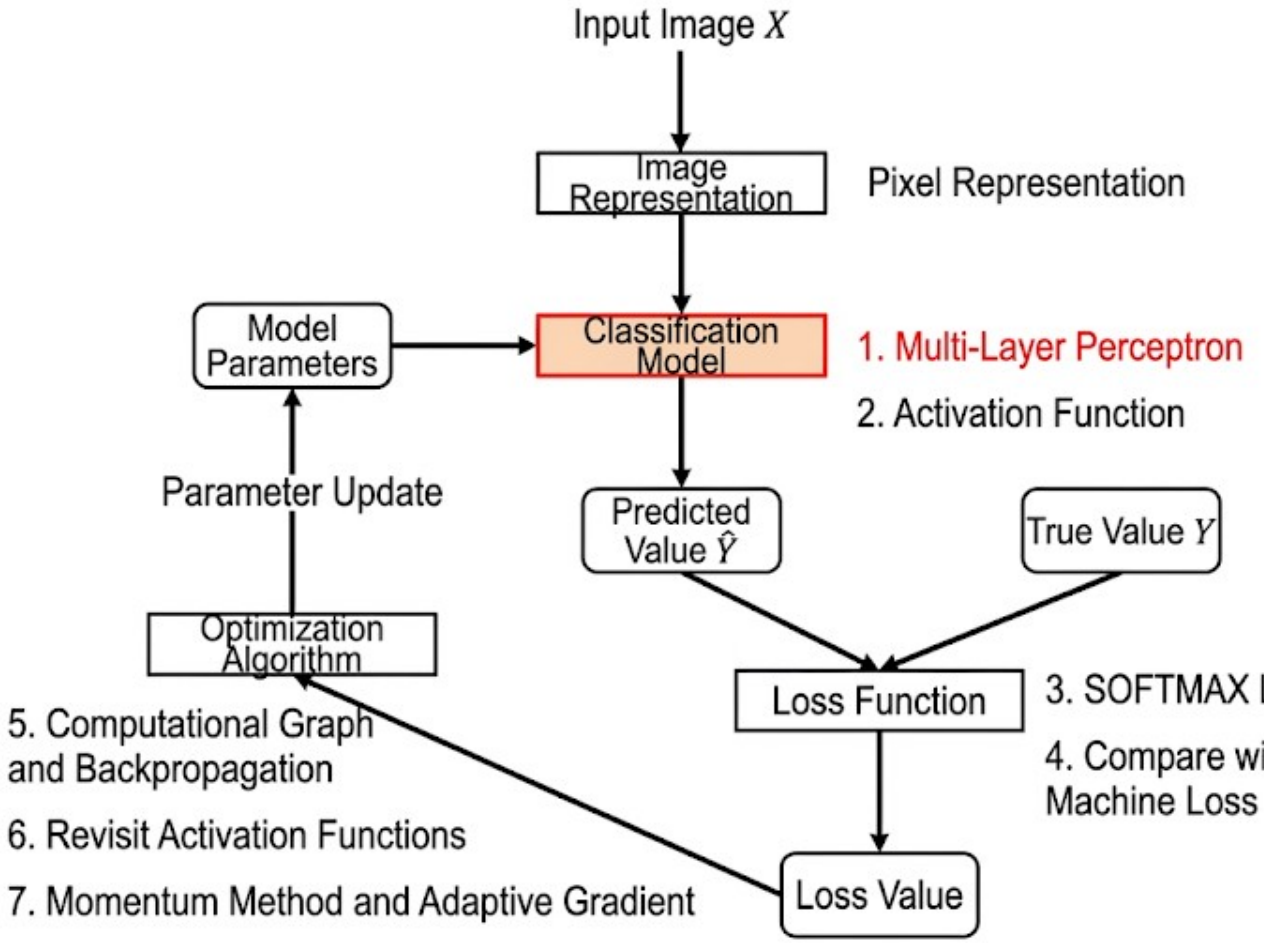
8. Weight Initialization
9. Batch Normalization
10. Underfitting, Overfitting & Dropout
11. Model Regularization & Hyperparameter Tuning

How to represent an image?

Use raw pixels as features, flattened into a column vector.



Each image in CIFAR-10 can be represented as a 3072 (32*32*3)-dimensional vector



Training Process

8. Weight Initialization
9. Batch Normalization
10. Underfitting, Overfitting and Dropout
11. Model Regularization and Hyperparameter Tuning

Linear Classifier

$$\boldsymbol{f}(\boldsymbol{x}, W) = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$$

Where, $\boldsymbol{x}$ represents the input image, its dimension is d;

$\boldsymbol{f}$ is the score vector, its dimension equals the number of classes c

$\boldsymbol{W} = [w_1 \quad \cdots \quad \boldsymbol{w_c}]^T$ is the weight matrix, $\boldsymbol{w_i} = [w_i \quad \cdots \quad w_{id}]^T$ is the weight vector for the i-th class

$\boldsymbol{b} = [b_1 \quad \cdots \quad b_c]^T$ is the bias vector, b_i is the bias for the i-th class

Fully Connected Neural Network

A fully connected neural network cascades multiple transformations to achieve the mapping from input to output.

Fully Connected Neural Network

2-Layer Fully Connected Network

$$f = W_2 max(0, W_1 x + b_1) + b_2$$

3-Layer Fully Connected Network

$$f = W_3 \, max(0, W_2 max(0, W_1 x + b_1) + b_2)$$

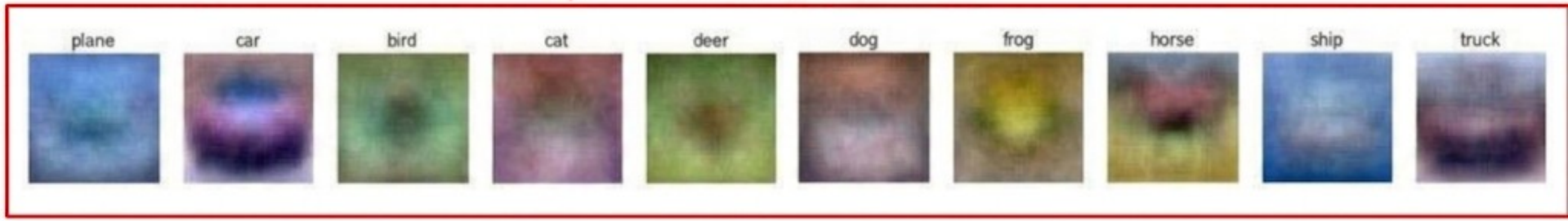
NOTE: Non-linear operations cannot be removed

Fully connected neural networks cascade multiple transformations to achieve input-to-output mapping.

Weights of Fully Connected Neural Networks

Linear Classifier: $f(x, W) = \boxed{W}x + b$

Weight Templates



Two-Layer Fully Connected Network:

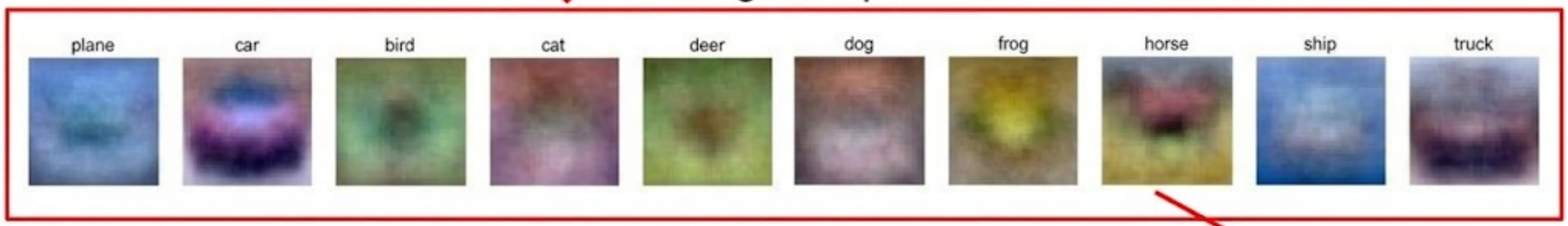
$$f = W_2 max(0, \boxed{W_1}x + b_1) + b_2$$

- In a linext classifier, W can be viewed as templats, which detersized by rnumber of classies.
- In a fully connected network:
  - W_1 can be viewed as templates; templates is specified.
  - W_2 integrates the templats of matchings to achieve the final classifiation score.

Fully Connected Neural Network Weights

Linear Classifier: $f(x, W) = Wx + b$

Weight Templates



2-Layer Fully Connected Network:

$$f = W_2 max(0, W_1 x + b_1) + b_2$$

Fully connected neural networks have stronger descriptive capability. Because adjusting the number of rows in W_1 is equivalent to increasing the number of templates, the classifier has the opportunity to learn templates for two horses in different directions.

Two horse heads

Fully Connected Neural Network and Linearly Inseparable

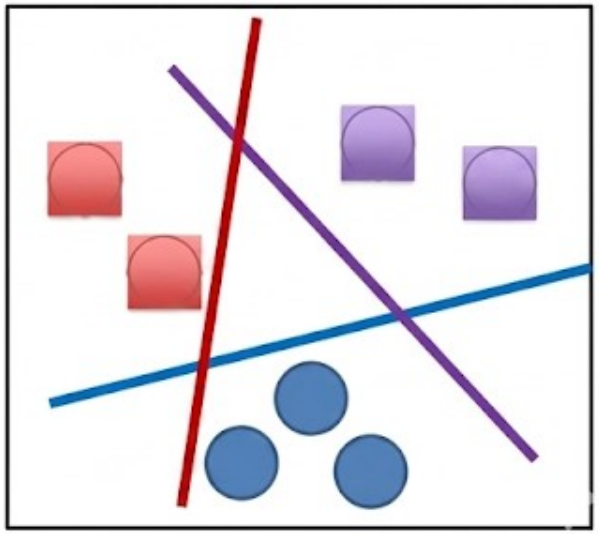
2-Layer Fully Connected Network

$$f = W_2 \max(0, W_1 x + b_1) + b_2$$

Linear Classifier

$$\mathbf{f(x, W) = Wx + b}$$

Linear Separability



Linearly Separable —— At least one linear boundary exists that can separate two classes without error.

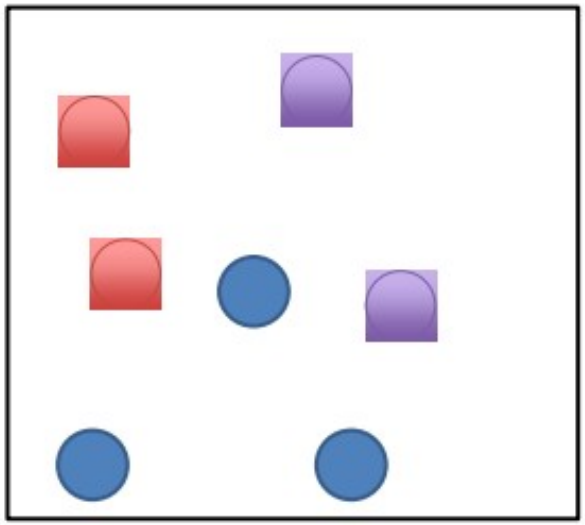
Fully Connected Neural Networks and Linearity and Non-separability

Two-Layer Fully Connected Network $f = W_2 \max(0, W_1 x + b_1) + b_2$

Linear Classifier $\mathbf{f(x, W) = Wx + b}$

Linear Separability —— An existing linear boundary can separate two classes of samples with errors.

Linear Inseparability



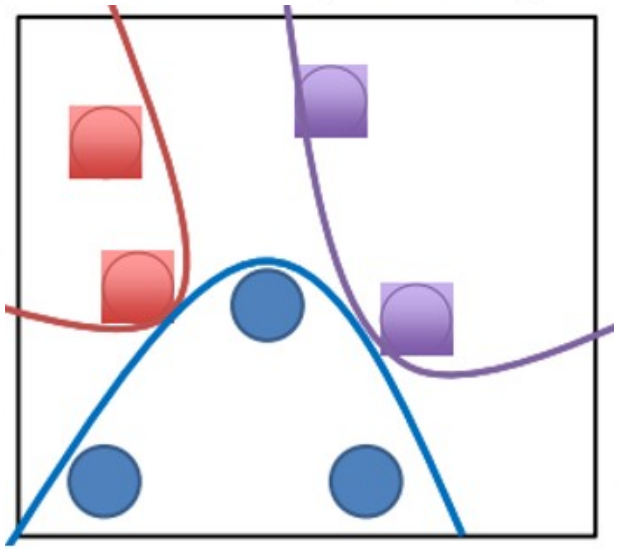
Fully Connected Neural Networks and Linearity and Non-separability

Two-Layer Fully Connected Network $f = W_2 \max(0, W_1 x + b_1) + b_2$

Linear Classifier $\mathbf{f(x, W) = Wx + b}$

Linear Separability —— An existing linear boundary can separate two classes of samples with errors.

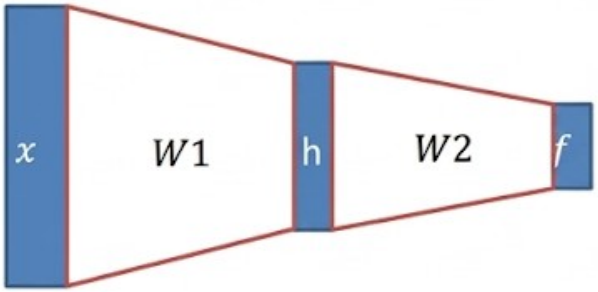
Linear Inseparability

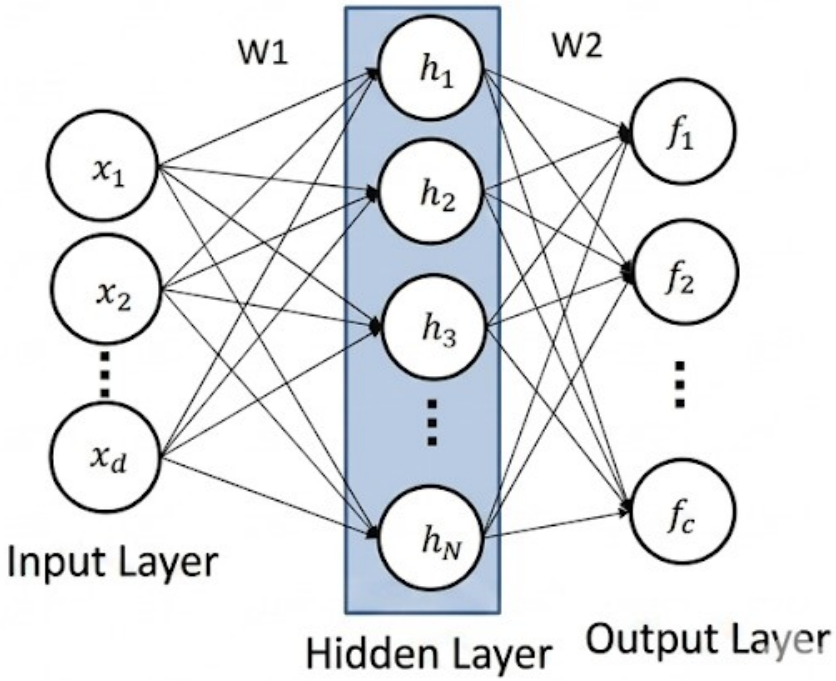


Fully Connected Neural Network Diagram and Naming

Two-layer Fully Connected Network

$$f = W_2 \max(0, W_1 x + b_1) + b_2$$

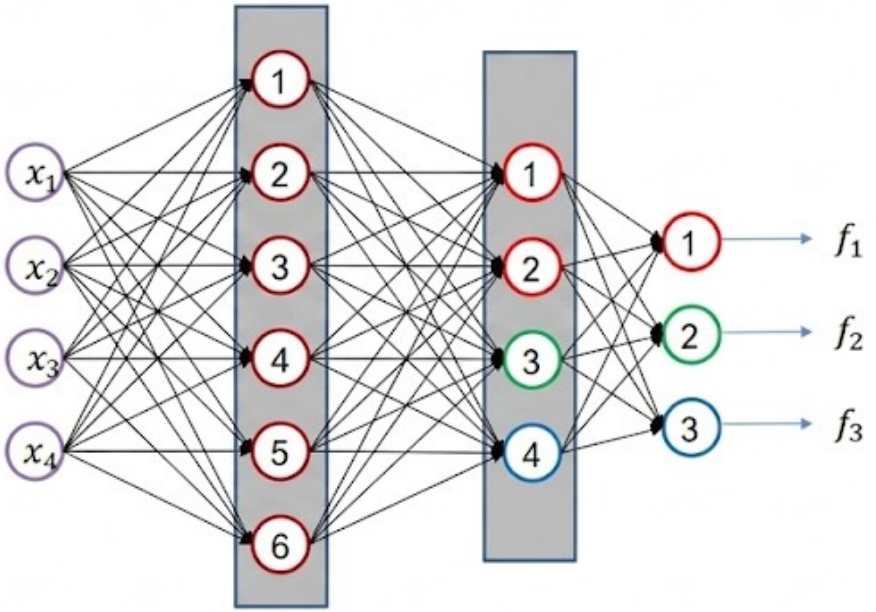




Fully Connected Neural Network Diagram and Naming

N-layer Fully Connected Neural Network —— A network with N layers **excluding* the input layer

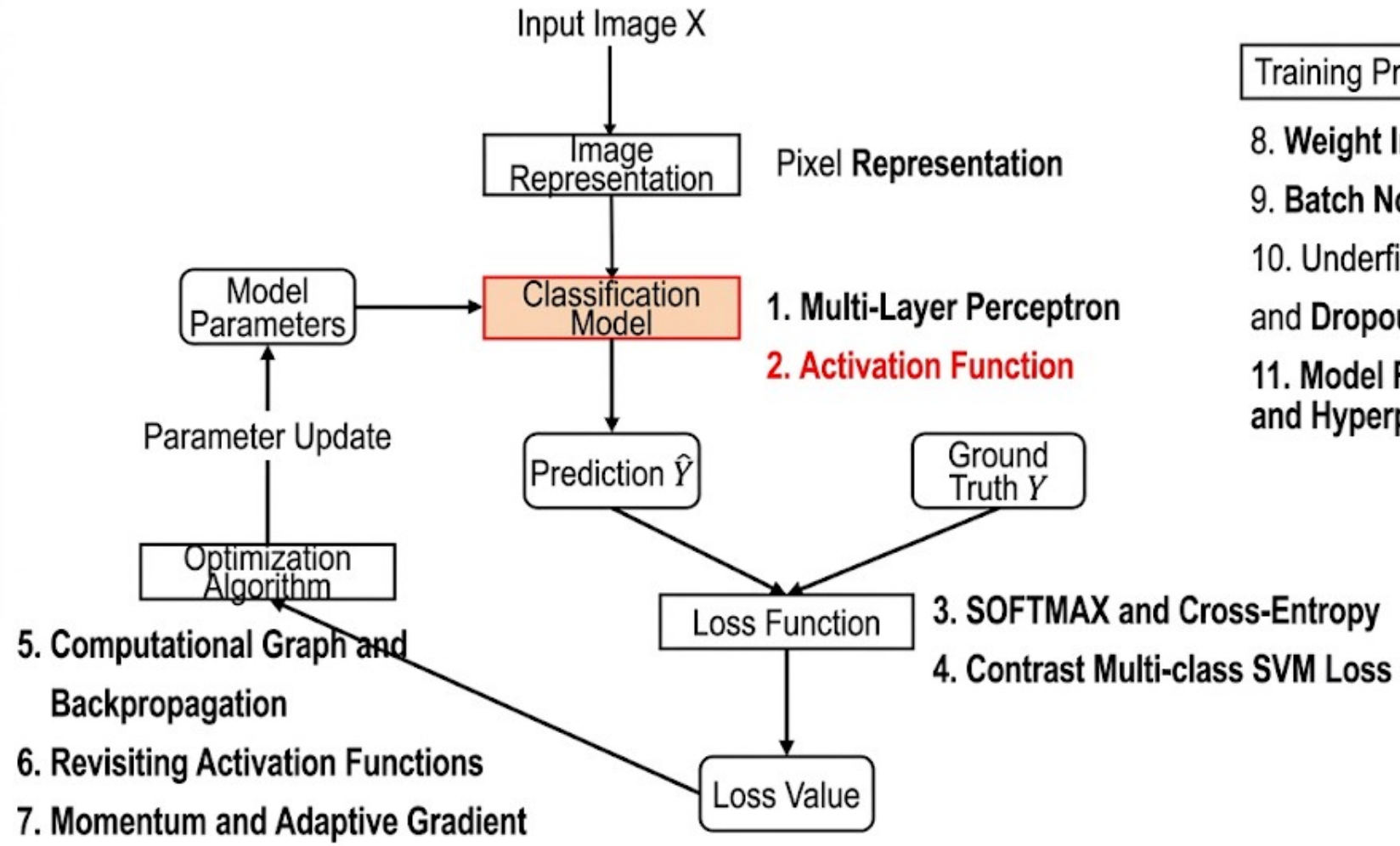
Fully Connected Neural Network with N Hidden Layers —— A network with N hidden layers



$$f = W_3 \max(0, W_2 \max(0, W_1 x + b_1) + b_2) + b_3$$

"3-layer Neural Network"

or "2-Hidden-Layer Neural Network"


Input Image X
Image Representation
Pixel Representation
Model Parameters
Classification Model
1. Multi-Layer Perceptron
2. Activation Function
Parameter Update
Prediction $\hat{Y}$
Ground Truth Y
Optimization Algorithm
Loss Function
3. SOFTMAX and Cross-Entropy
4. Contrast Multi-class SVM Loss
5. Computational Graph and Backpropagation
6. Revisiting Activation Functions
7. Momentum and Adaptive Gradient
Loss Value
Training Process
8. Weight Initialization
9. Batch Normalization
10. Underfitting, Overfitting, and Dropout
11. Model Regularization and Hyperparameter Tuning

Activation Functions

- Why do we need non-linear operations?

Activation Functions

Three-Layer Fully Connected Network

$$f = W_3 \max(0, W_2 \max(0, W_1 x + b_1) + b_2) + b_3$$

Ans: Without activation functions in the network, the fully connected neural network will become a linear classifier.

Removing Activation Functions

$$f = W_3(W_2(W_1 x + b_1) + b_2) + b_3$$

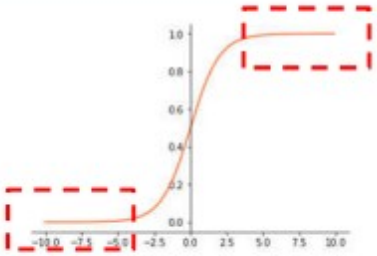
$$= W_3 W_2 W_1 x + (W_3 W_2 b_1 + W_3 b_2 + b_3)$$

$$= W' x + b'$$

Commonly used activation functions

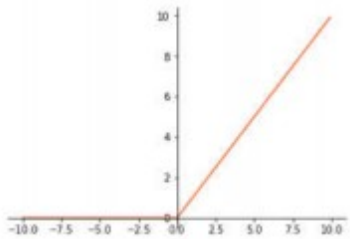
Sigmoid

$1/(1+e^{-x})$



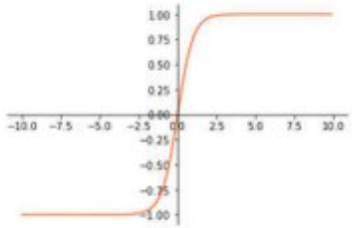
ReLU

$\max(0, x)$



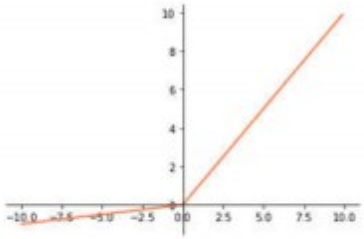
tanh

$$\frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$$



Leaky ReLU

$\max(0.1x, x)$



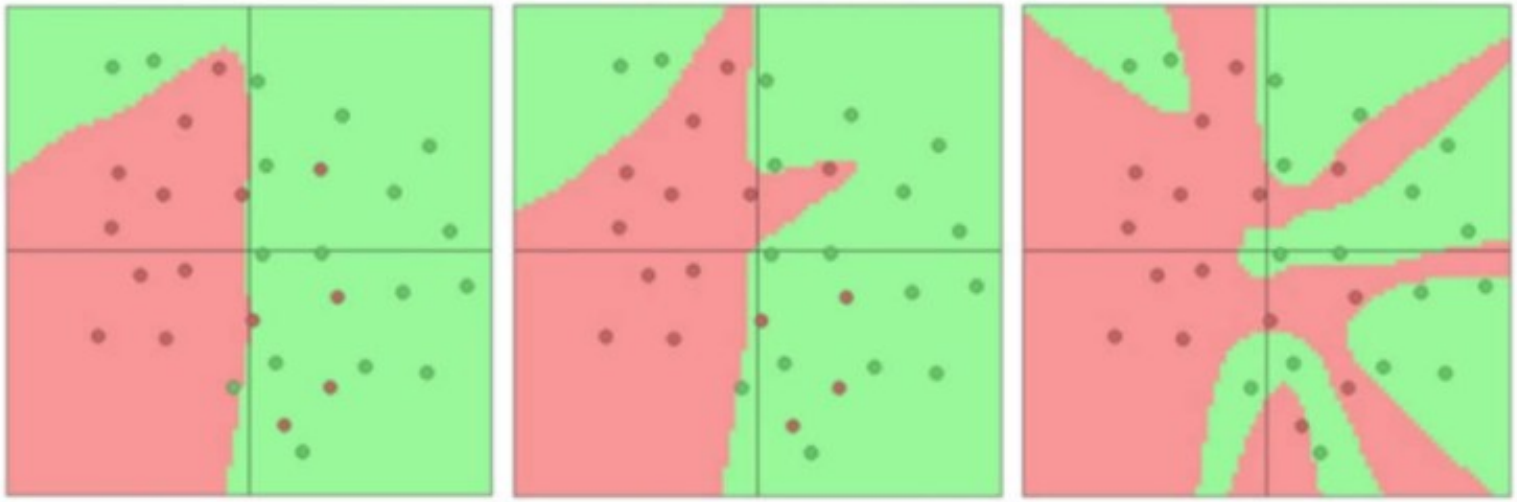
Network Structure Design

1. Whether to use hidden layers, and whether to use one or multiple hidden layers? (Depth design)

2. How many neurons should be set in each hidden layer? (Width design)

There is no unified answer!

Network structure design



3 hidden layer neurons 6 hidden layer neurons 20 hidden layer neurons

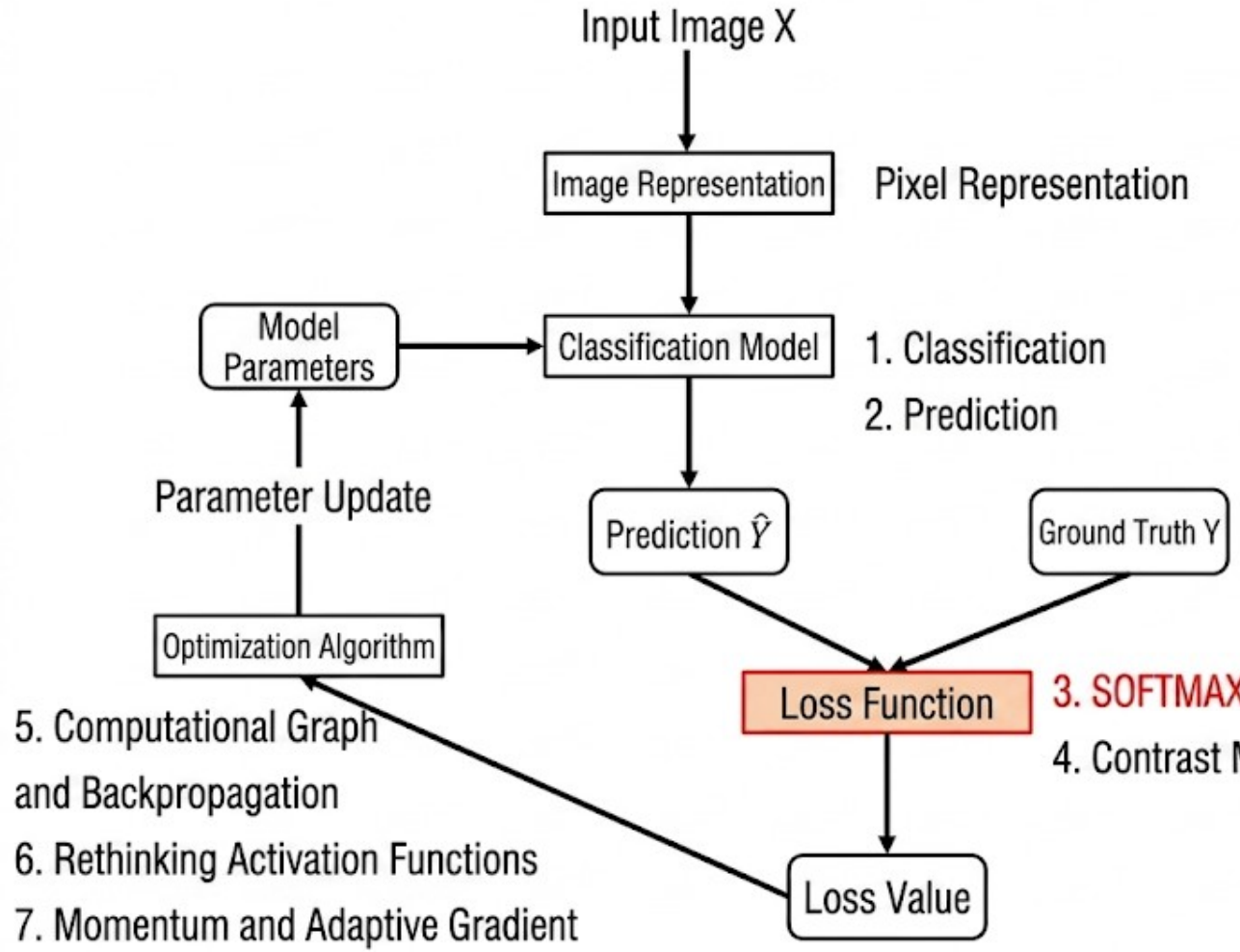
Conclusion: The more neurons there are, the more complex the decision boundary can be, and the stronger the classification ability on this set becomes.

Network structure design

Adjust the complexity of the neural network model based on the difficulty of the classification task. The more difficult the classification task, the deeper and wider the neural network structure we design should be. However, it is important to note that the fully connected neural network model with the highest classification accuracy on the training set may not necessarily have the best recognition performance in real-world scenarios (overfitting).

Summary of Fully Connected Neural Networks

- Composition of a fully connected neural network: one input layer, one output layer, and multiple hidden layers.
- The number of neurons in the input and output layers is determined by the task, while the number of hidden layers and the number of neurons in each hidden layer need to be manually specified.
- The activation function is an important part of a fully connected neural network; without it, the fully connected neural network would degenerate into a linear classifier.



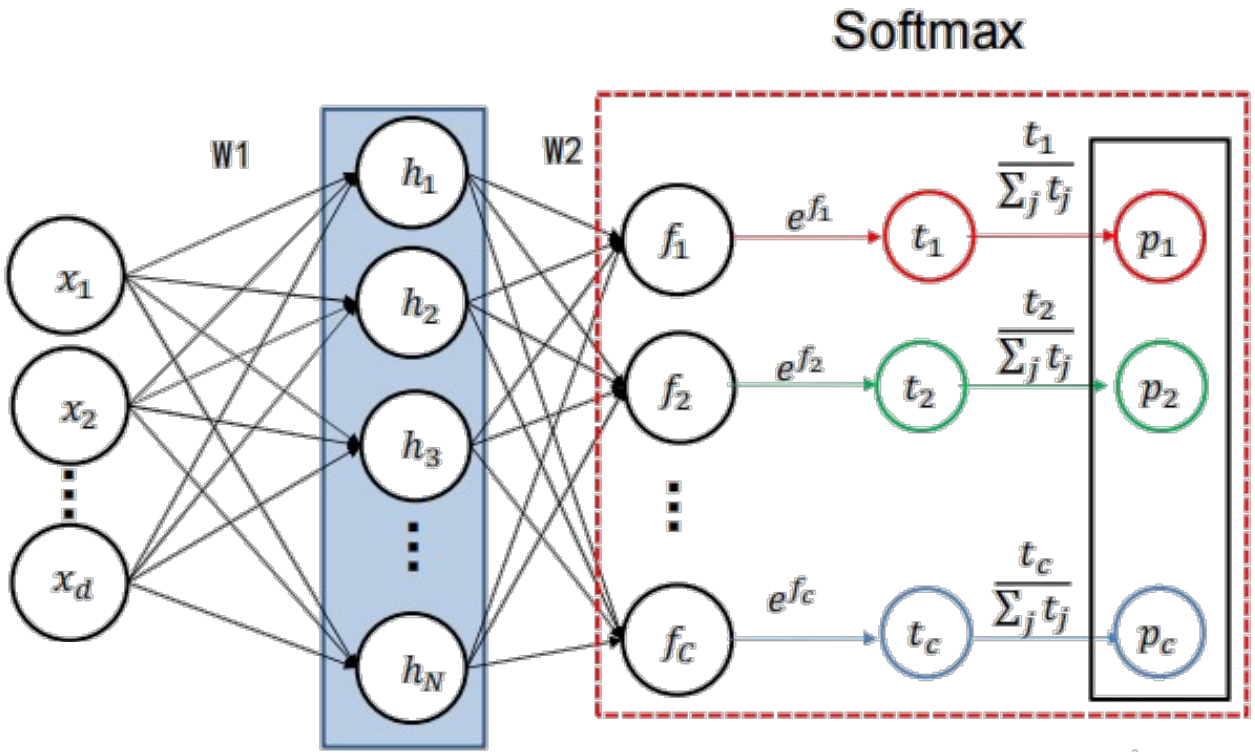
Training Process

8. Weight Initialization
9. Batch Normalization
10. Underfitting, Overfitting, and Dropout
11. Model Regularization and Hyperparameter Tuning

SOFTMAX

Two-layer fully connected network

$$f = W_2 \max(0, W_1 x + b_1) + b_2$$



SOFTMAX

- Example

$f = W_2 \max(0, W_1 x + b_1) + b_2$

Scores	ef_i	$\frac{t_i}{\sum_j t_j}$	
0.6	**1.81**	**0. 21**	bird
-2.3	0.10	0. 01	cat
1.9	6.69	0. 78	car

Exponential

Normalization

Image

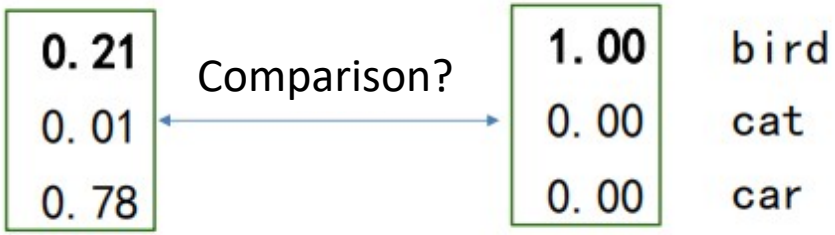
Scores

Normalization

Probabilities

Cross-entropy loss

- How to measure the distance between the current classifier output and the predicted value?



Classifier predicted distribution q(x)

True distribution p(x)

Cross-Entropy

Entropy: $H(p) = -\sum_x p(x) \log p(x)$

Cross-Entropy: $H(p,q) = -\sum_x p(x) \log q(x)$

Relative Entropy: $KL(p||q) = -\sum_x p(x) \log \frac{q(x)}{p(x)}$

Also known as KL divergence. It is used to measure the dissimilarity between two distributions.

The relationship between the three:

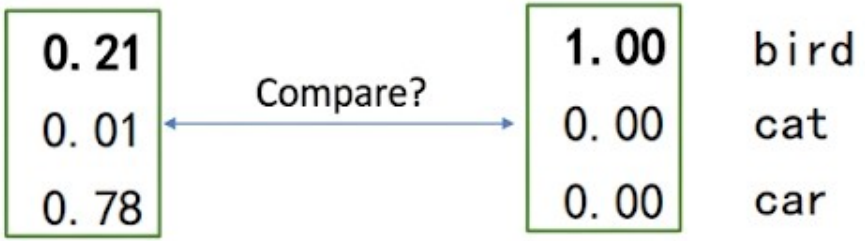
$$H(p,q) = -\sum_x p(x) \log q(x)$$

$$= -\sum_x p(x) \log p(x) - \sum_x p(x) \log \frac{q(x)}{p(x)} \text{ (preserved)}$$

$$= H(p) + KL(p||q)$$

Cross-Entropy Loss

- How to measure the distance between the current classifier output and the ground truth values?



Classifier Predicted Distribution $q(x)$

Ground Truth Distribution $p(x)$

Note: When the ground truth distribution is in one-hot form, the cross-entropy loss simplifies to:

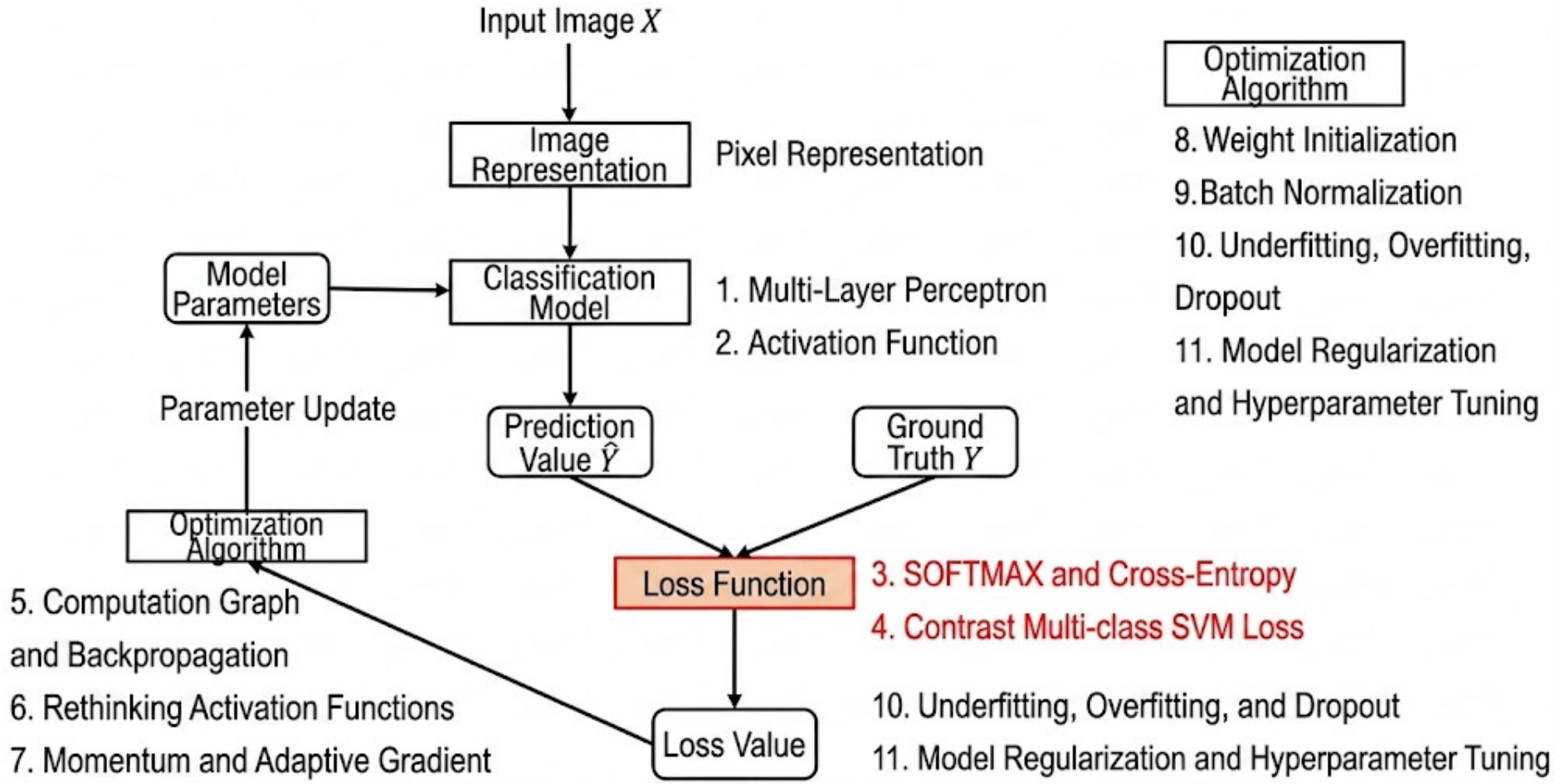
$L_i = -\log(q_j)$, where j is the ground truth class

How to measure the difference between two random distributions

Cross-En $H(p,q) = -\sum_{i=1}^{C} \mathrm{p}(x_i)\log\big(q(x_i)\big)$

Cross-Entropy Loss:

$$L_i = -\sum_{i=1}^{C} \mathrm{p}(x_i)\log\big(q(x_i)\big)$$


Input Image X
Image Representation
Pixel Representation
Model Parameters
Classification Model
1. Multi-Layer Perceptron
2. Activation Function
Parameter Update
Prediction Value $\hat{Y}$
Ground Truth Y
Optimization Algorithm
Loss Function
3. SOFTMAX and Cross-Entropy
4. Contrast Multi-class SVM Loss
Loss Value
Optimization Algorithm
8. Weight Initialization
9. Batch Normalization
10. Underfitting, Overfitting, Dropout
11. Model Regularization and Hyperparameter Tuning
5. Computation Graph and Backpropagation
6. Rethinking Activation Functions
7. Momentum and Adaptive Gradient
10. Underfitting, Overfitting, and Dropout
11. Model Regularization and Hyperparameter Tuning

Cross-Entropy Loss vs. Multi-class SVM Loss

Image

Classifier

0.6
-2.3
1.9

Scores f

Multi-class SVM Loss

$$L_i = \sum_{j \neq y_i} \max\left(0, s_j - s_{y_i} + 1\right)$$

$$= \max(-2.3 - 0.6 + 1, 0) + \max(1.9 - 0.6 + 1, 0)$$

$$= 2.3$$

Cross-Entropy Loss

Cross-Entropy Loss

e^{f_i}

1.81
0.10
6.69

Exponential

Normalize

$\frac{t_i}{\sum_j t_j}$

0. 21
0. 01
0. 78

Probabilities

$$L_i = -\log\left(\frac{e^{sy_i}}{\sum_j e^{s_j}}\right)$$

=0.92

Cross-Entropy Loss vs. Multiclass SVM Loss

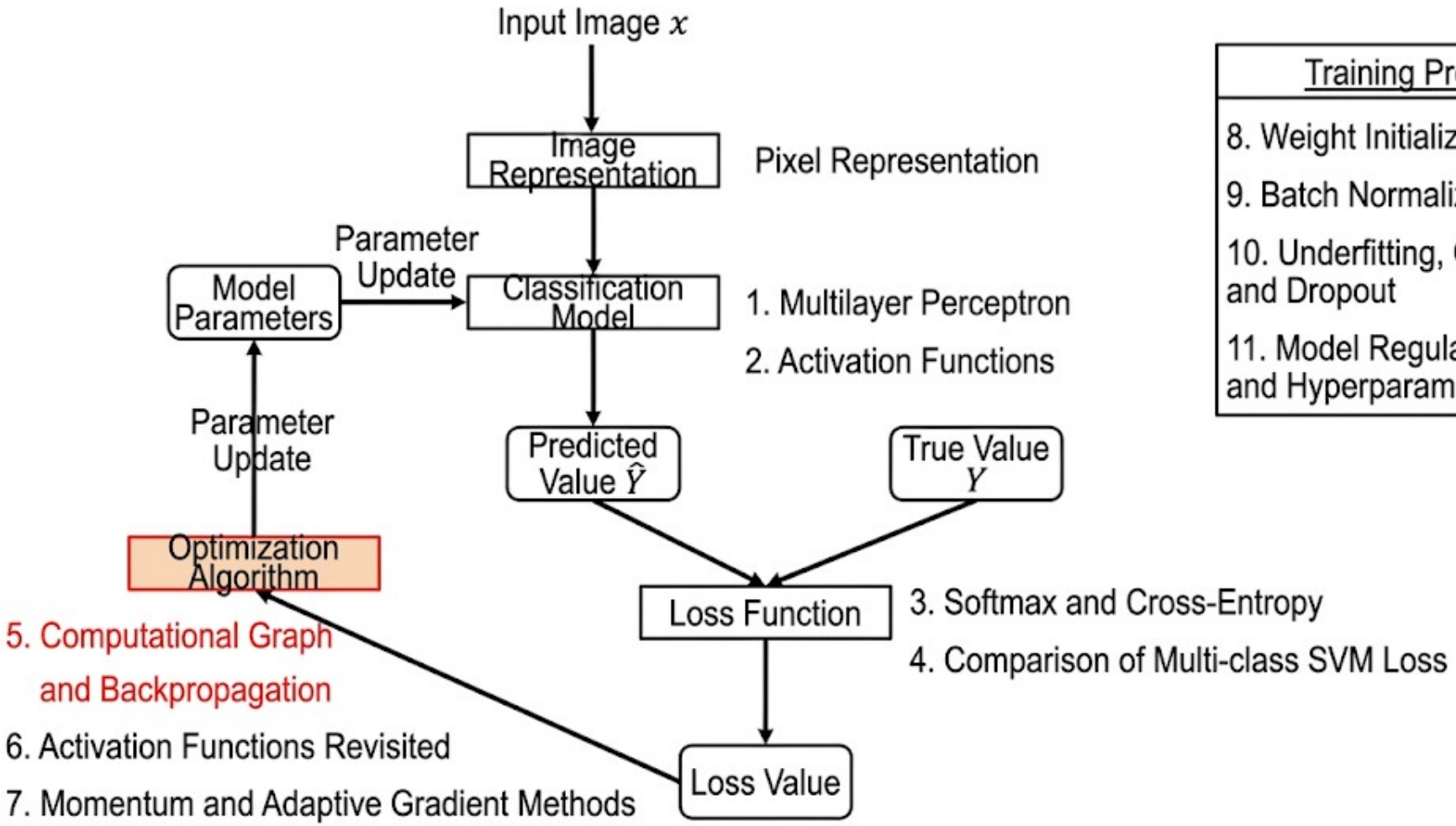
Assume the score f

$y_i = 0$

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

$$L_i = \sum_{j \neq y_i} \max\left(0, s_j - s_{y_i} + 1\right)$$

[10, -2, 3]	0.0004	0
[10, 9, 9]	0.2395	0
[10, -100, -100]	1.5×10^{-48}	0



Training Process

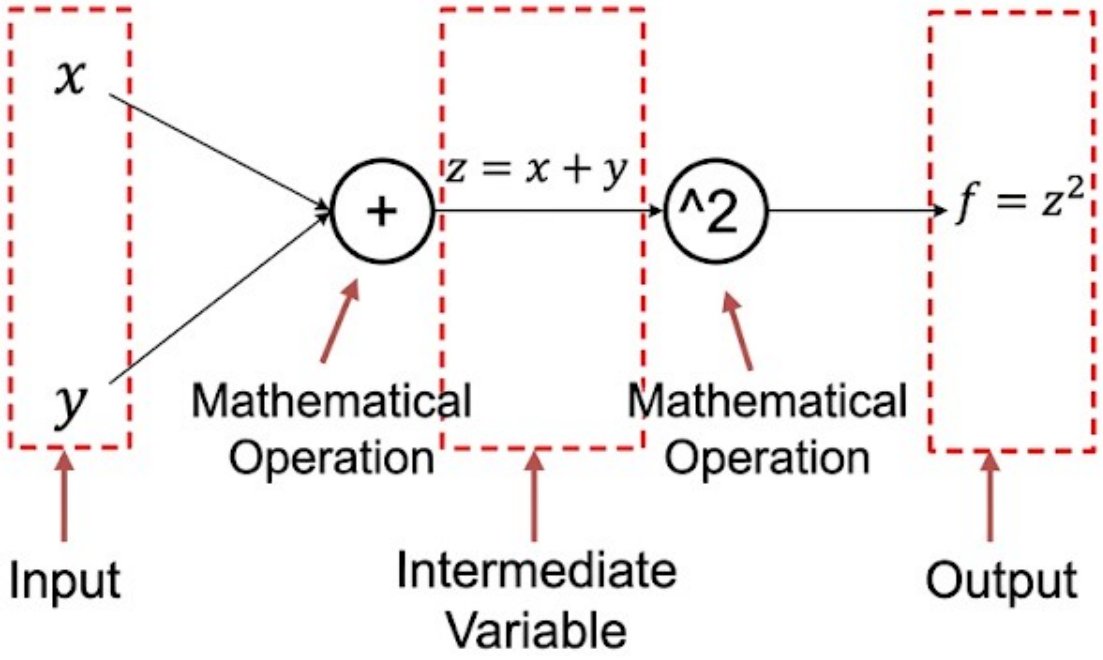
8. Weight Initialization
9. Batch Normalization
10. Underfitting, Overfitting, and Dropout
11. Model Regularization and Hyperparameter Tuning

What is a computational graph?

A computational graph is a directed graph used to express the computational relationships among inputs, outputs, and intermediate variables. Each node in the graph corresponds to a mathematical operation.

Computational Graph and Backpropagation Algorithm

Computational Graph of the function $f = (x + y)^2$

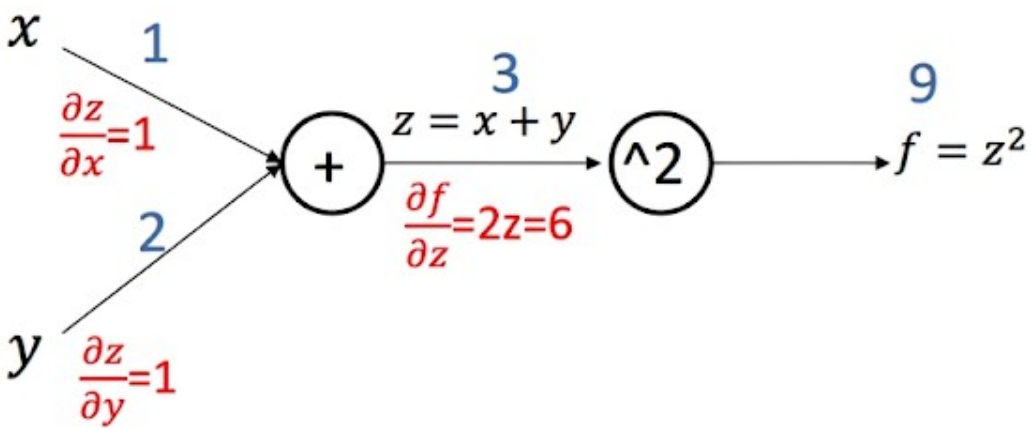


Forward and Backward Computation of Computational Graphs

Computational Graph of function $f = (x + y)^2$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial z}\frac{\partial z}{\partial x}$$

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial z}\frac{\partial z}{\partial y}$$



Computational Graph Summary

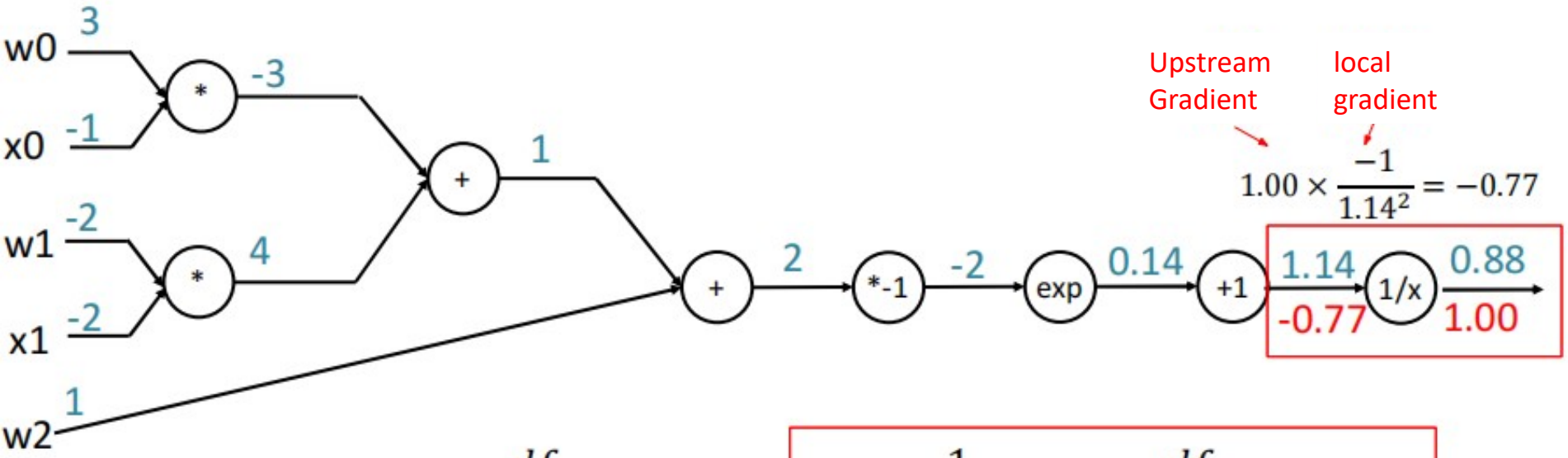
- Any arbitrarily complex function can be represented in the form of a computational graph.
- Throughout the entire computational graph, each gate unit receives some inputs and then performs the following two calculations:

  a) The output value of this gate

  b) The local gradient of its output with respect to its input.
- Using the chain rule, the gate unit should multiply the backpropagated gradient by its local gradient with respect to its input, thereby obtaining the gradient of the entire network's output with respect to each input value of that gate unit.

Backpropagation Example 2

$$f(w,x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

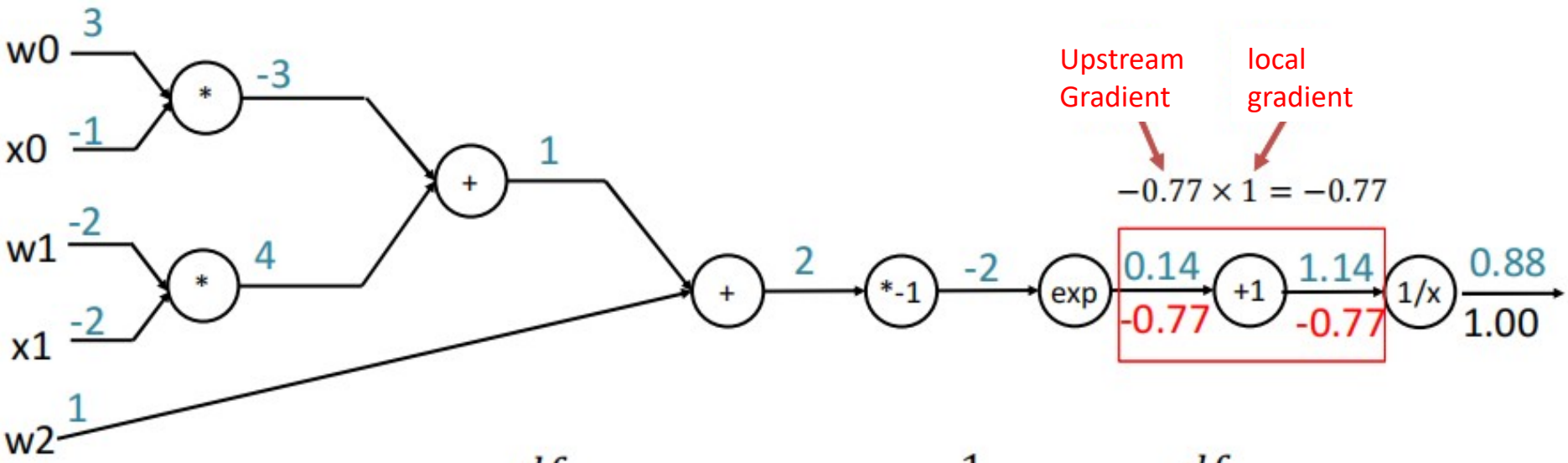
$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Backpropagation Example 2

$$f(w,x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

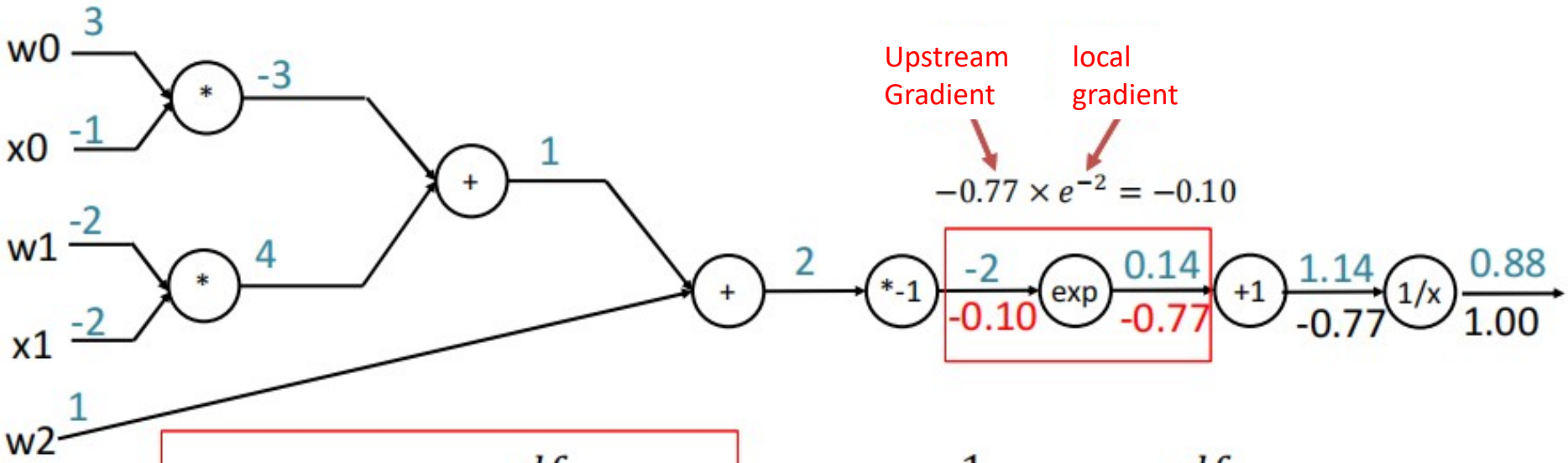
$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Backpropagation Example 2

$$f(w,x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

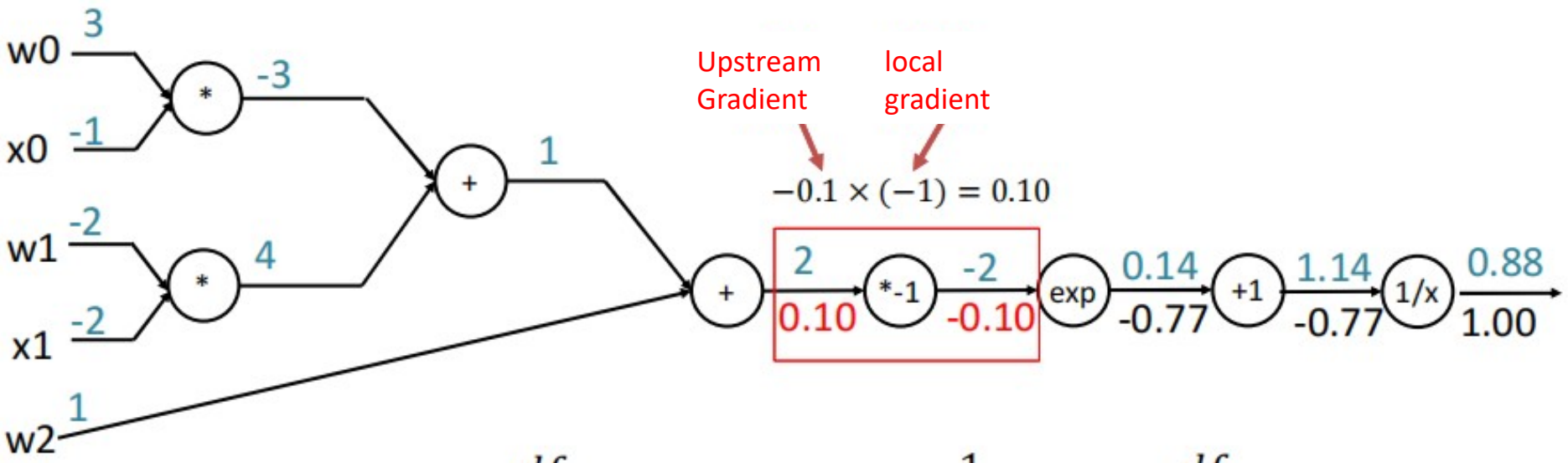
$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Backpropagation Example 2

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

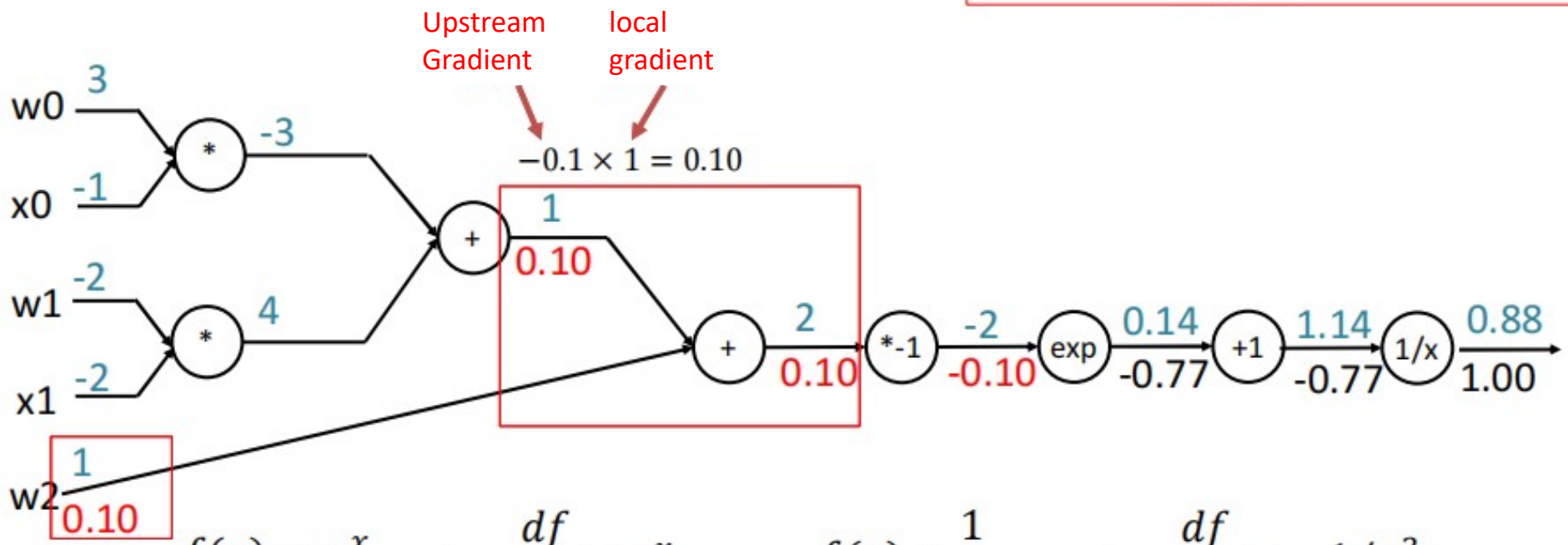
$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Backpropagation Example 2

$$f(w,x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$

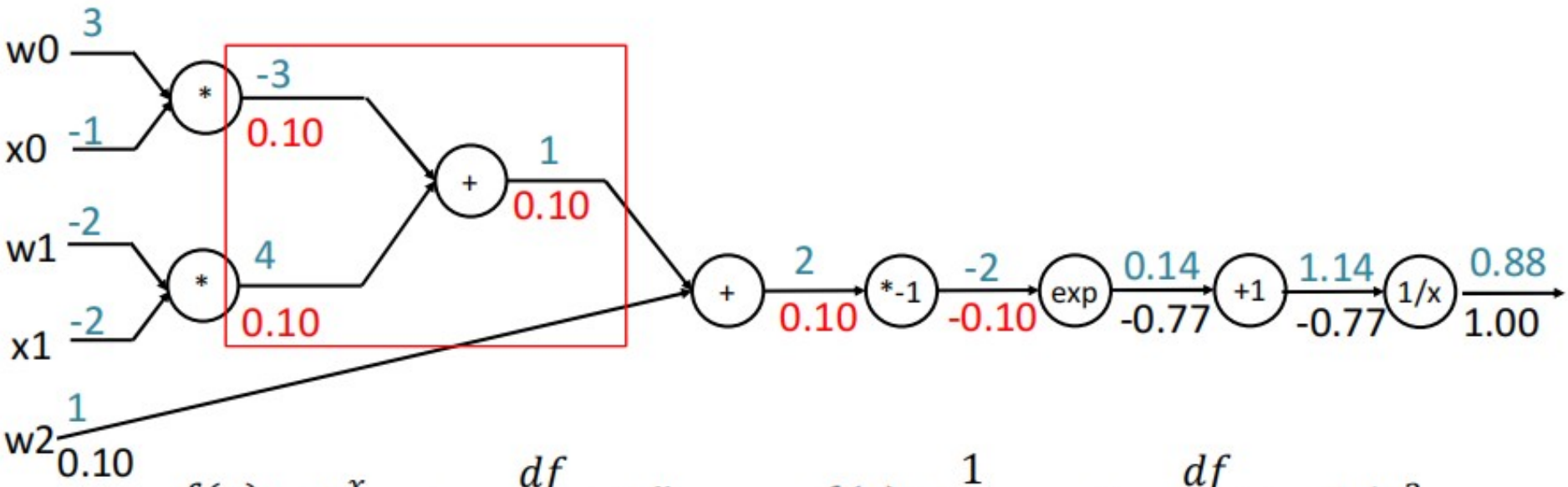


$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x \qquad f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a \qquad f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Backpropagation Example 2

$$f(w,x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$

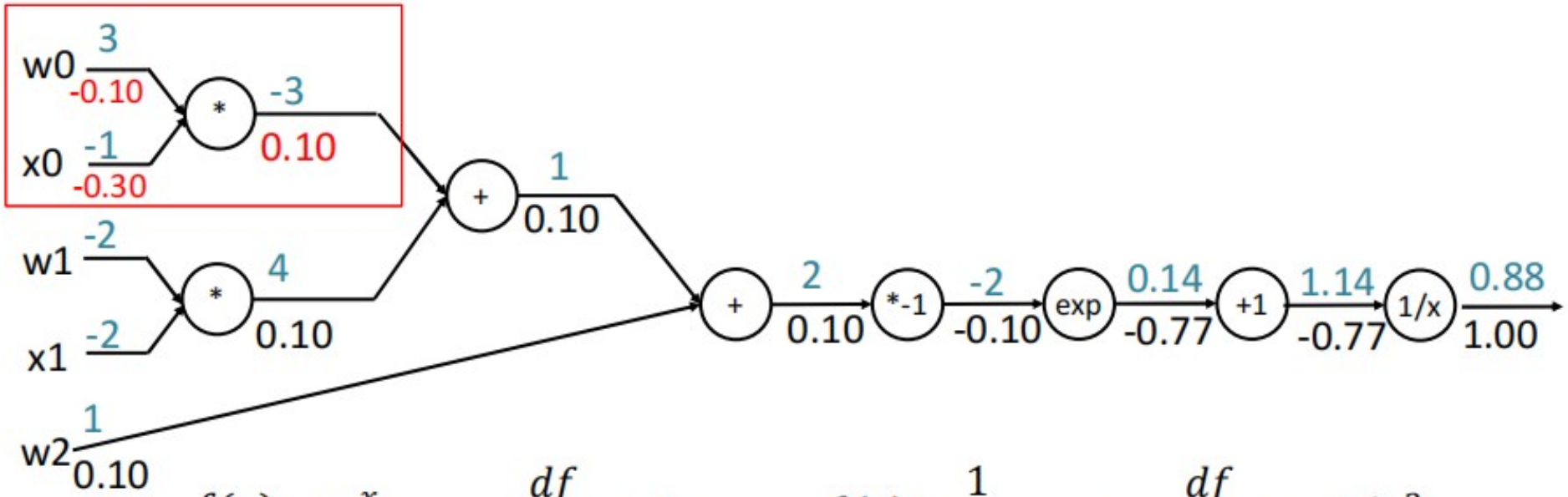


$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x \qquad f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a \qquad f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Backpropagation Example 2

$$f(w,x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$

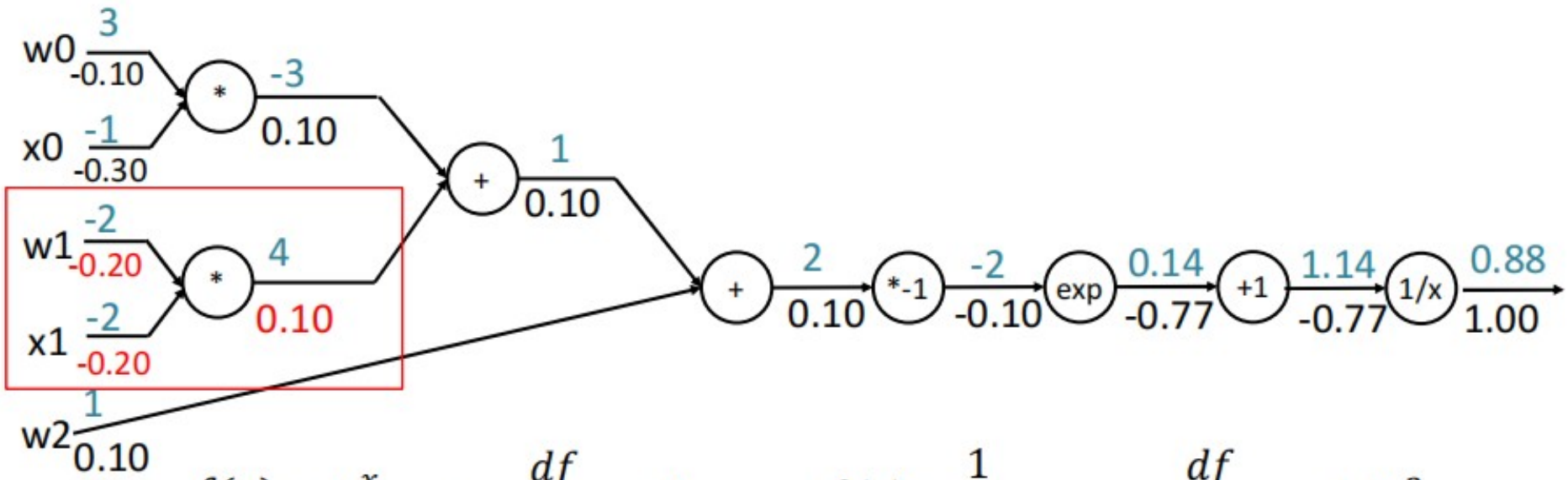


$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x \qquad f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a \qquad f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Backpropagation Example 2

$$f(w,x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$

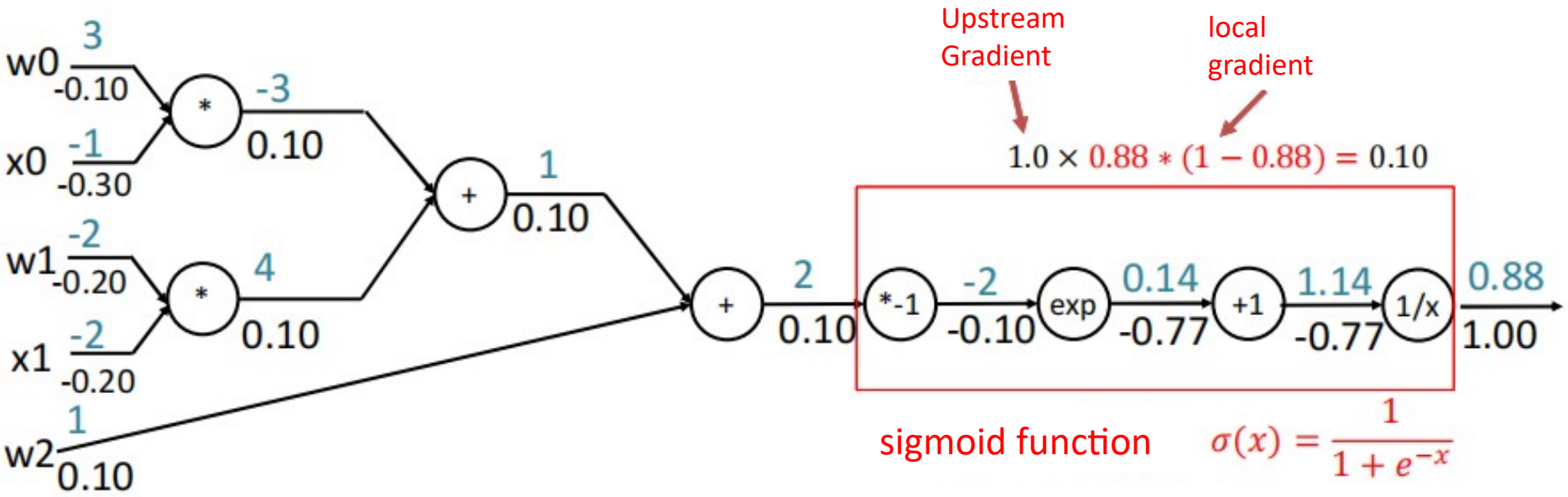


$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x \qquad f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a \qquad f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Granularity of the computation graph

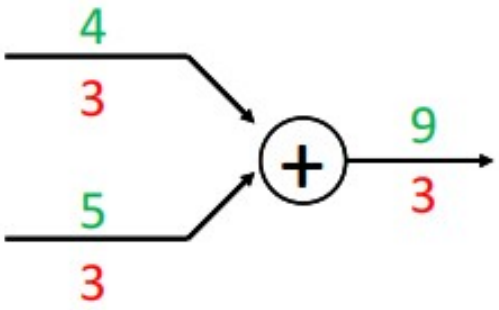
$$f(w,x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



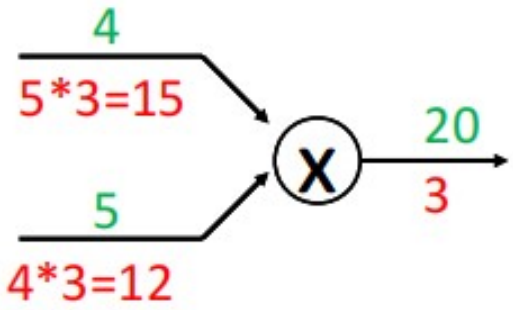
$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}}\right)\left(\frac{1}{1 + e^{-x}}\right) = (1 - \sigma(x))\sigma(x)$$

Common gate units in computational graphs

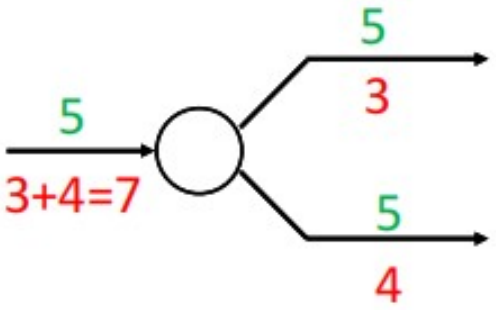
Addition Gate



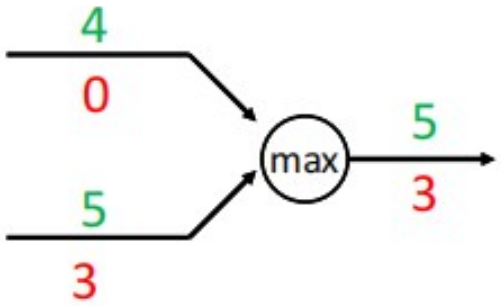
Multiplication Gate

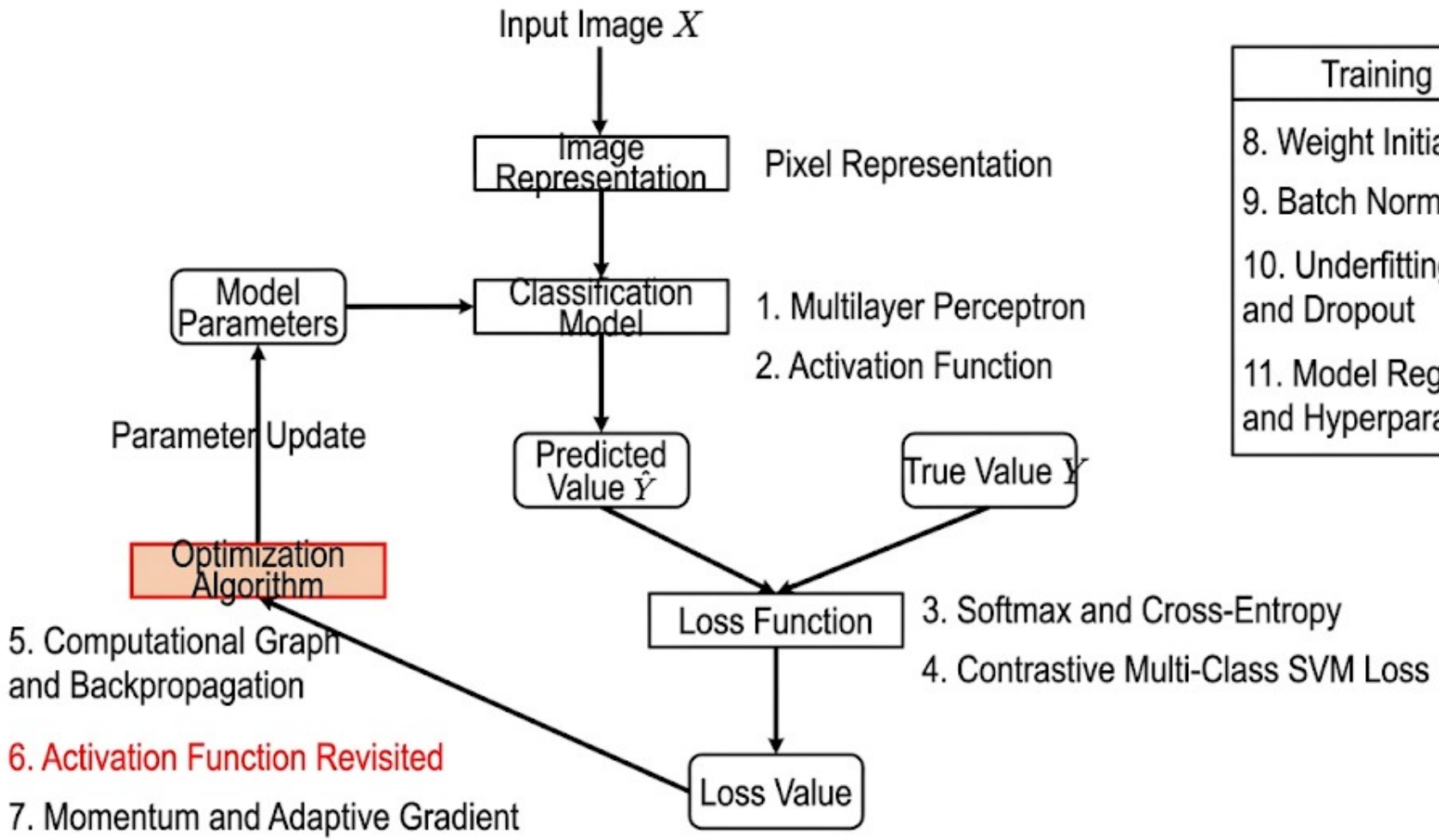


Copy Gate



Max Gate

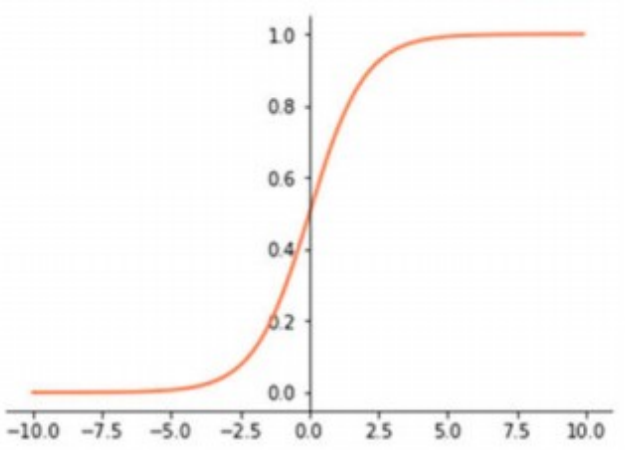




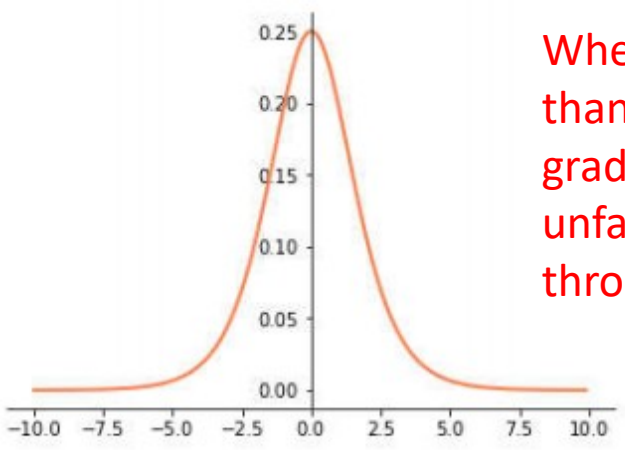
Training Process
8. Weight Initialization
9. Batch Normalization
10. Underfitting, Overfitting, and Dropout
11. Model Regularization and Hyperparameter Tuning

Revisit the activation function

Sigmoid activation function $\sigma(x) = 1/(1 + e^{-x})$



sigmoid function

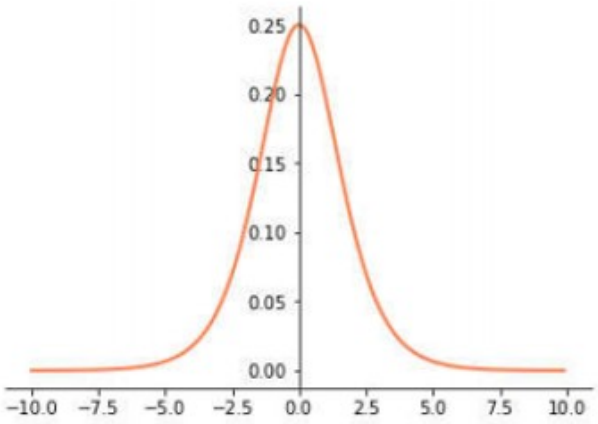


Derivative of the sigmoid function

When the input value is greater than 10 or less than -10, the local gradient is 0; this is very unfavorable for the gradient flow through the network.

Gradient Vanishing

Gradient vanishing is a very critical problem in neural network training, and its essence is caused by the multiplicative nature of the chain rule.

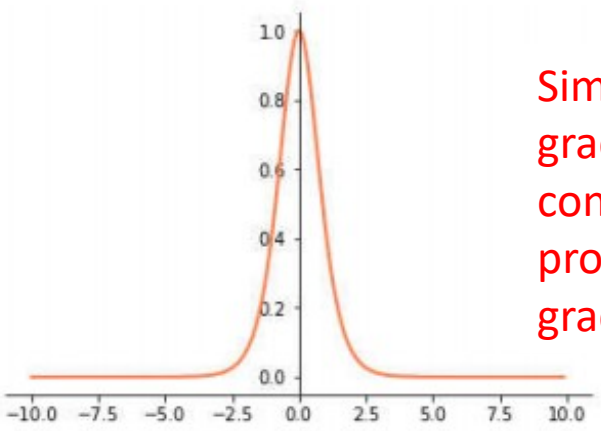


Derivative of the sigmoid function

Activation Function

Hyperbolic tangent activation function

$$\tanh(x) = \frac{\sinh x}{\cosh x} = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$$

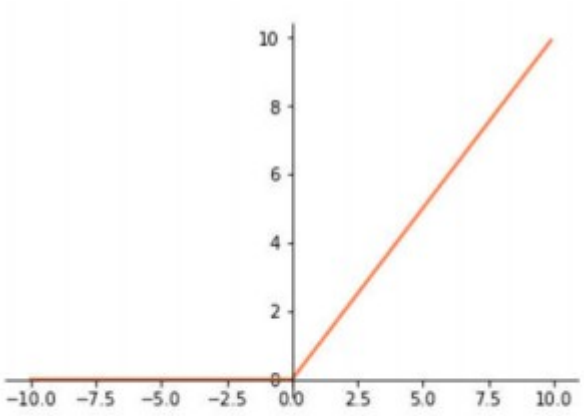


Similar to sigmoid, the local gradient characteristics are not conducive to the backward propagation of the network gradient flow.

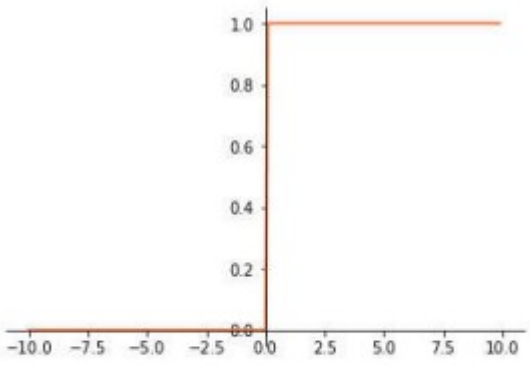
The derivative function of the tanh function

Activation function selection

ReLU activation function: $f(x) = \max(0, x)$



ReLU function

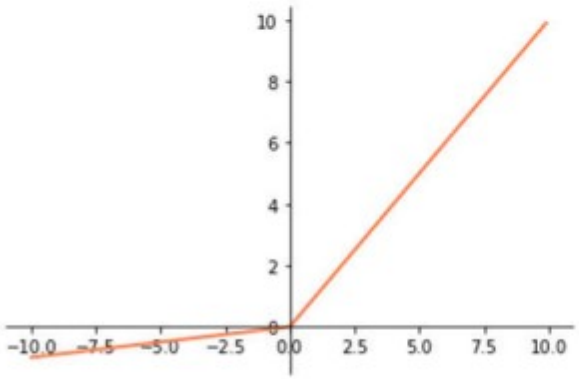


Derivative of ReLU function

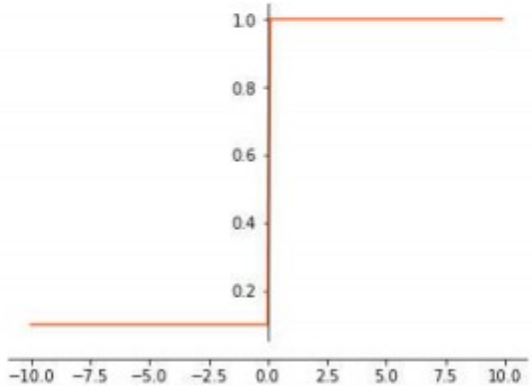
When the input is greater than 0, the local gradient will never be 0, which is relatively beneficial for the propagation of gradient flow.

Activation function selection

Leakly ReLU activation function: $f(x) = \max(0.01x, x)$



Leaky ReLU function

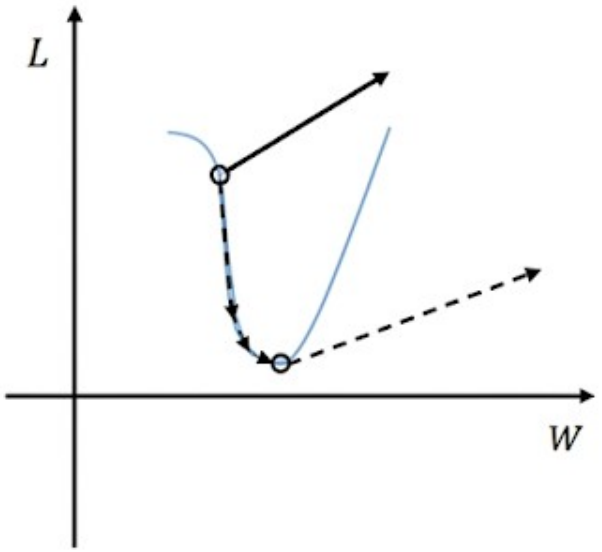


Derivative of ReLU function

There is almost no "dead zone," meaning the gradient will never be 0. The reason for saying "almost" is that the function has no derivative at 0.

Exploding Gradient

- Exploding gradients are also caused by the multiplicative property of the chain rule.

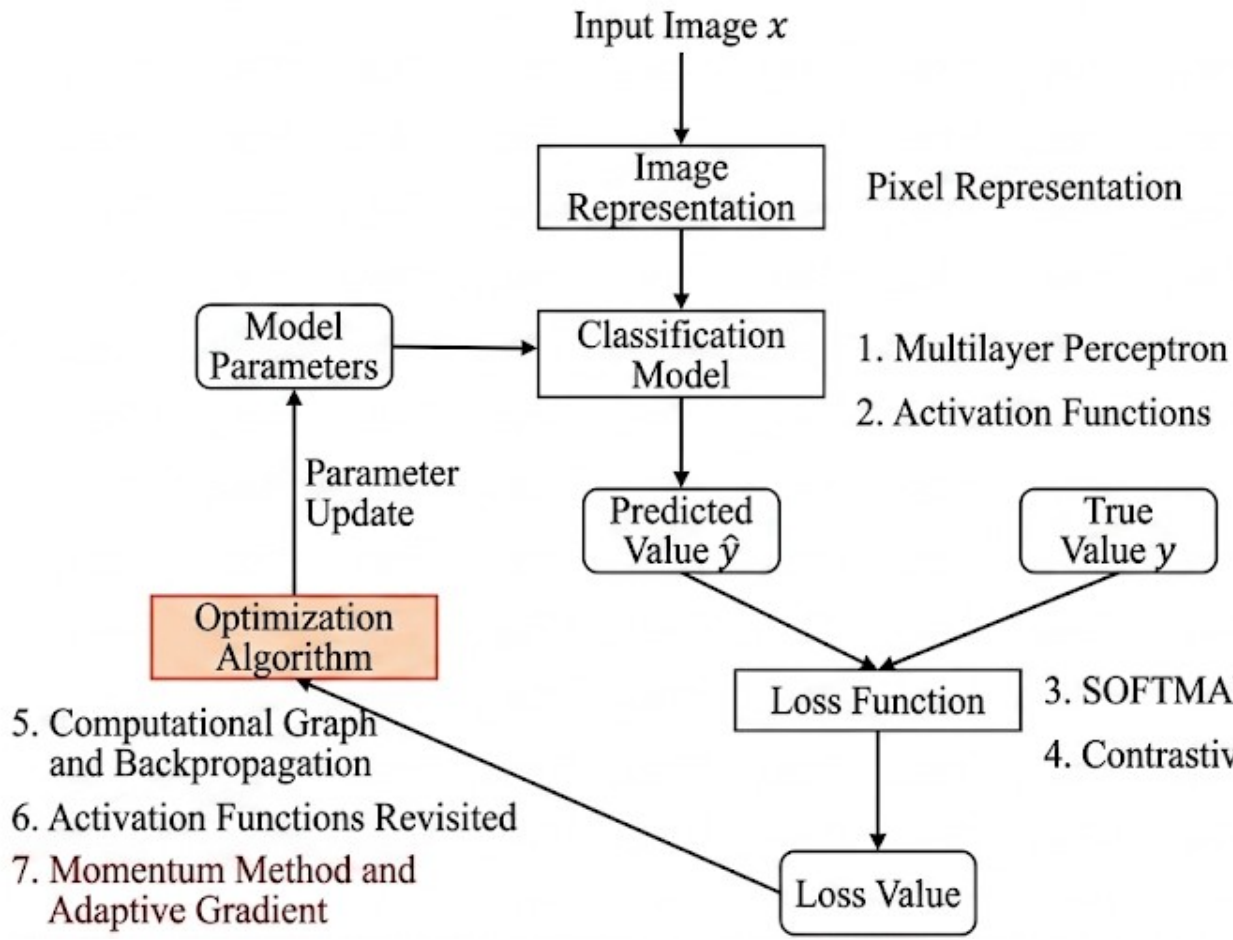


Exploding Gradient: After multiplying moving the gradient by the learning rate at a cliff edge, it will be a very large value, which ' "flies" out of the reasonable region, ultimately leading to non-convergence;

Solution: Limiting to the pranmec dreciкind the step size along the gradient direction to be within a certain value can avoid "flying" out, and this method is also called Gradient Clipping.

Summary of Activation Function Selection

- Whenever possible, choose the ReLU function or the Leaky ReLU function. Compared to Sigmoid/tanh,
- the ReLU or Leaky ReLU function allows gradients to flow more smoothly, resulting in faster convergence during training.



Training Process

8. Weight Initialization
9. Batch Normalization
10. Underfitting, Overfitting, and Dropout
11. Model Regularization and Hyperparameter Tuning

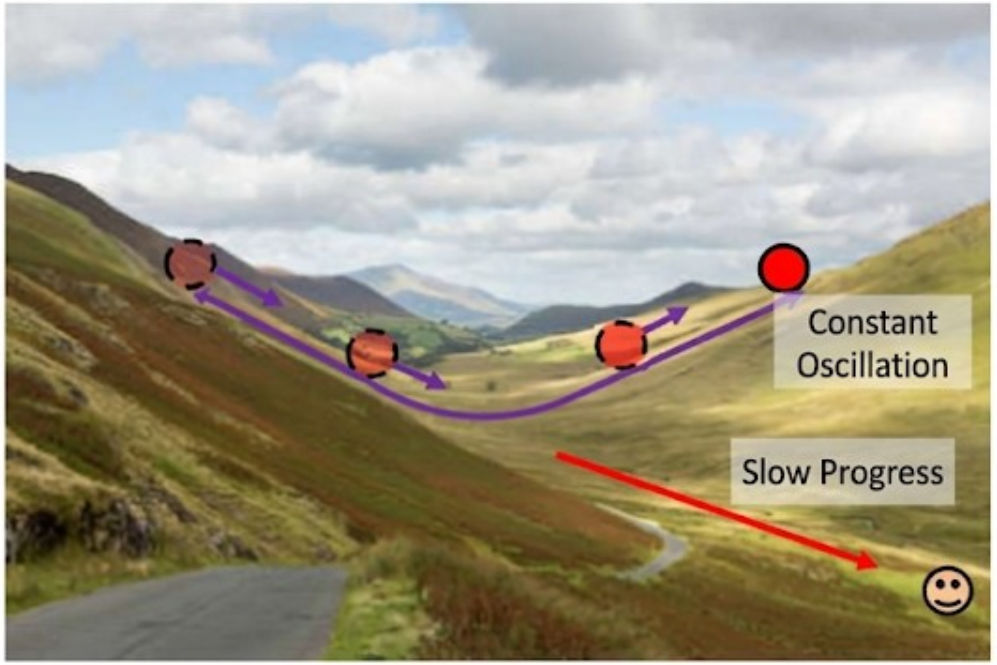
Gradient Algorithm Improvement

- Problems with Gradient Descent Algorithm
- Momentum Method
- Adaptive Gradient and RMSProp
- ADAM
- Summary

Problems in Gradient Descent Method

- **Loss function characteristics:** changes rapidly in one direction while changing slowly in another.
- **Optimization goal**: walk from the starting point to the bottom smiley face.
- **Problem with Gradient Descent:** oscillations between the mountainside, slow progress towards the valley direction.

Simply increasing the step size won't speed up the algorithm's convergence rate!



Momentum Method

Why is it effective?
During accumulation, oscillation directions cancel each other out, while flat directions are reinforced.

- **Goal:** To improve problems with the gradient descent algorithm, specifically to reduce oscillation and accelerate movement towards the valley.
- **Improvement Idea:** Utilize cumulative historical gradient information to update gradients.

Minibatch Gradient Descent with Momentum

Require: learning rate ϵ, momentum coefficient μ

Require: initial parameter θ

Initialize velocity $v = 0$

while termination criterion not met **do:**

1. Sample m examples (batch size) $\{x^{(1)}, \ldots, x^{(m)}\}$ from the training set with corresponding targets $\{y^{(1)}, \ldots, y^{(m)}\}$
2. Compute gradient: $g \leftarrow \frac{1}{m} \nabla_\theta \sum_i L\left(f(x^{(i)}; \theta), y^{(i)}\right)$
3. Update velocity: $v = \mu v + (1 - \mu) g$
4. Update parameter: $\theta \leftarrow \theta - \epsilon v$

end while

μ value range [0.1)
μ=0 is equivalent to the gradient descent algorithm
Recommended setting: 0.9

Minibatch Gradient Descent

Require: learning rate ϵ

Require: initial parameter θ

while termination criterion not met **do:**

1. Sample m examples (batch size) $\{x^{(1)}, \ldots, x^{(m)}\}$ from the training set with corresponding targets $\{y^{(1)}, \ldots, y^{(m)}\}$
2. Compute gradient: $g \leftarrow \frac{1}{m} \nabla_\theta \sum_i L\left(f(x^{(i)}; \theta), y^{(i)}\right)$
3. Update parameter: $\theta \leftarrow \theta - \epsilon g$

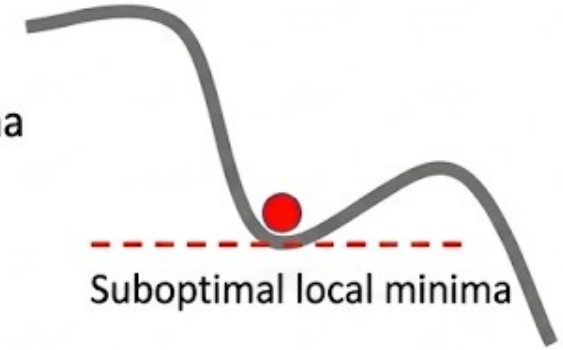
end while

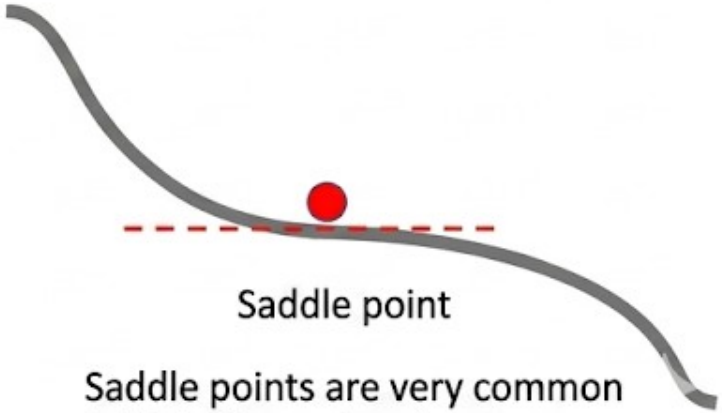
What else can the Momentum method do?

Phenomenon: Loss functions often have suboptimal local minima or saddle points (very common in high-dimensional spaces)

Problem with gradient descent algorithm: At local minima and saddle points,
the gradient is 0, and the algorithm cannot pass through.

Advantages of the Momentum method: Due to the presence of momentum, the algorithm can jump out of suboptimal local minima and saddle points, finding a better solution.



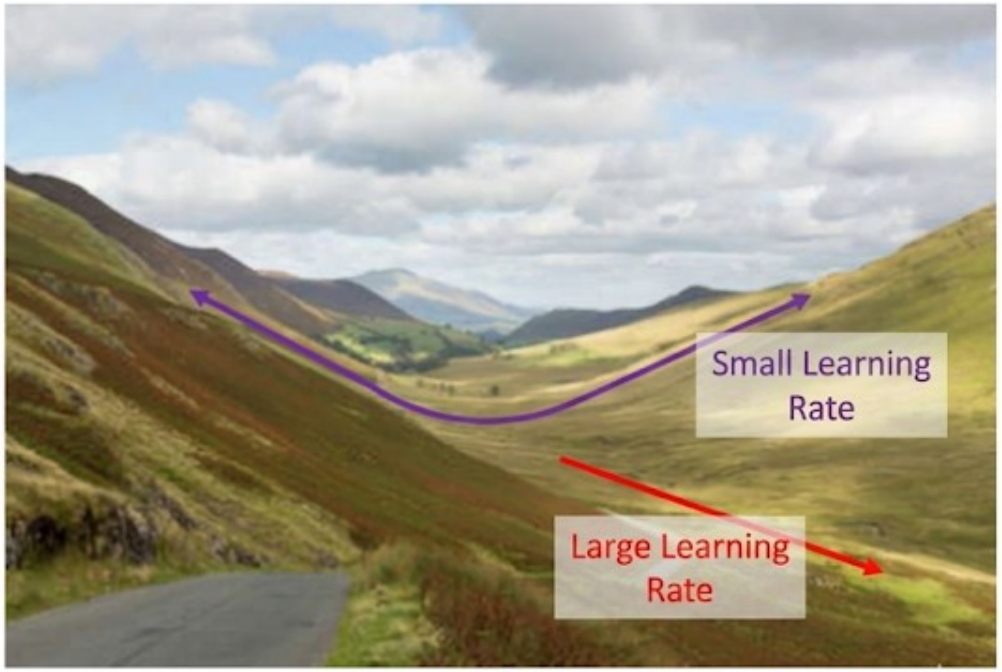


Saddle points are very common in high-dimensional spaces

Adaptive Gradient Method

- Adaptive Gradient Method reduces oscillation and accelerates progress to the valley floor by reducing the step size in oscillating directions and increasing the step size in flat directions;
- How to distinguish oscillating directions from flat directions?

  Answer: The direction with a large squared gradient magnitude is the oscillating direction; The direction with a small squared gradient magnitude is the flat direction.



AdaGrad

The AdaGrad method is an adaptive gradient method

AdaGrad

Require: learning rate ϵ, decay rate ρ

Require: initial parameters θ

Require: small constant δ, for numerical stability when dividing by small numbers. (Normally set to 10^{-5})

Initialize accumulation variable $r = 0$

while stopping criterion not satisfied do:

1. Sample a minibatch of m samples $\{x^{(1)}, \ldots, x^{(m)}\}$ from the training set with corresponding targets $\{y^{(1)}, \ldots, y^{(m)}\}$
2. Compute gradient: $g \leftarrow \frac{1}{m} \sum_i \nabla_\theta L\left(f\left(x^{(i)}; \theta\right), y^{(i)}\right)$
3. Accumulate squared gradient: $r \leftarrow r + g * g$
4. Update parameters: $\theta \leftarrow \theta - \frac{\epsilon}{\sqrt{r+\delta}} g$

end while

Mini-Batch Gradient Descent Algorithm

Require: learning rate ϵ

Require: initial parameters θ

while stopping criterion not satisfied do:

1. Sample a minibatch of m samples $\{x^{(1)}, \ldots, x^{(m)}\}$ from the training set with corresponding targets $\{y^{(1)}, \ldots, y^{(m)}\}$
2. Compute gradient: $g \leftarrow \frac{1}{m} \sum_i \nabla_\theta L\left(f\left(x^{(i)}; \theta\right), y^{(i)}\right)$
3. Update parameters: $\theta \leftarrow \theta - \epsilon g$

end while

RMSProp

The RMSProp method is an adaptive gradient method.

RMSProp

Require: learning rate ϵ, decay rate ρ

Require: initial parameters θ

Require: small constant δ, used for numerical stability when dividing by a small value. (Normally set to 10^{-5})

Initialize accumulation variable $r = 0$

while stopping criterion not satisfied **do**:

1. Sample a minibatch of m samples $\{x^{(1)}, \dots, x^{(m)}\}$ from training set with corresponding targets $\{y^{(1)}, \dots, y^{(m)}\}$
2. Compute gradient: $g \leftarrow \frac{1}{m} \sum_i \nabla_\theta L(f(x^{(i)}; \theta), y^{(i)})$
3. Accumulate squared gradient: $r \leftarrow \rho r + (1 - \rho) g \odot g$
4. Update parameters: $w \leftarrow w - \frac{\epsilon}{\sqrt{r + \delta}} g$

end while

ρ range [0, 1)
ρ= 0: only consider the strength of the current gradient
Recommended setting: 0.999

Mini-Batch Gradient Descent Algorithm

...rs θ

...rion not satisfied **do**:

1. Sample a minibatch of m samples $\{x^{(1)}, \dots, x^{(m)}\}$ from training set with corresponding targets $\{y^{(1)}, \dots, y^{(m)}\}$
2. Compute gradient: $g \leftarrow \frac{1}{m} \sum_i \nabla_\theta L(f(x^{(i)}; \theta), y^{(i)})$
3. Update parameters: $\theta \leftarrow \theta - \epsilon g$

end while

Adam

Adam Algorithm

Require: learning rate ϵ, decay rate ρ, momentum coefficient μ (Recommended values 0.999, are Recommended 0.9 respectively)

Require: initial parameter θ

Require: small constant δ for numerical stability when dividing by by small numbers. (Usually set to 10^{-5})

Initialize cumulative variables $r = 0$, $v = 0$

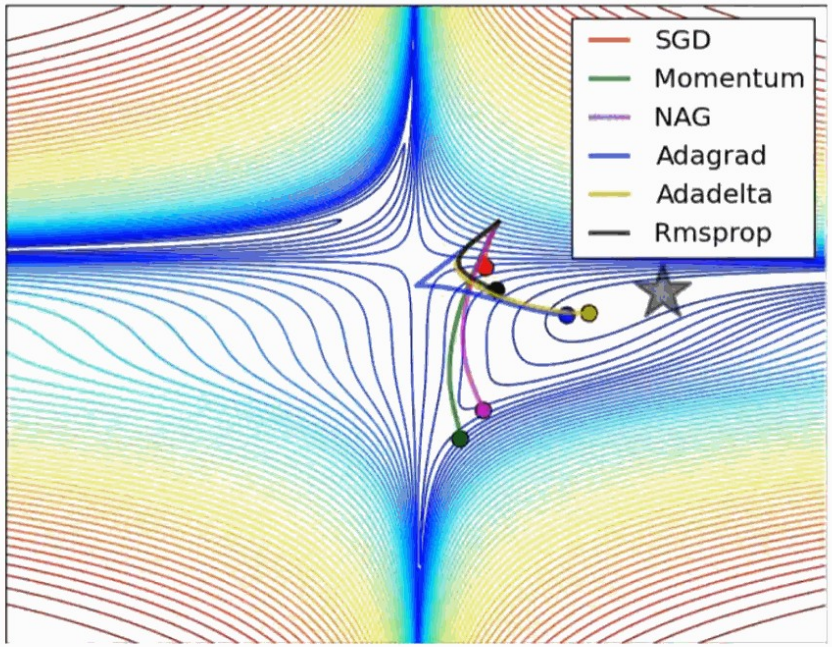
while stopping criterion is not met do:

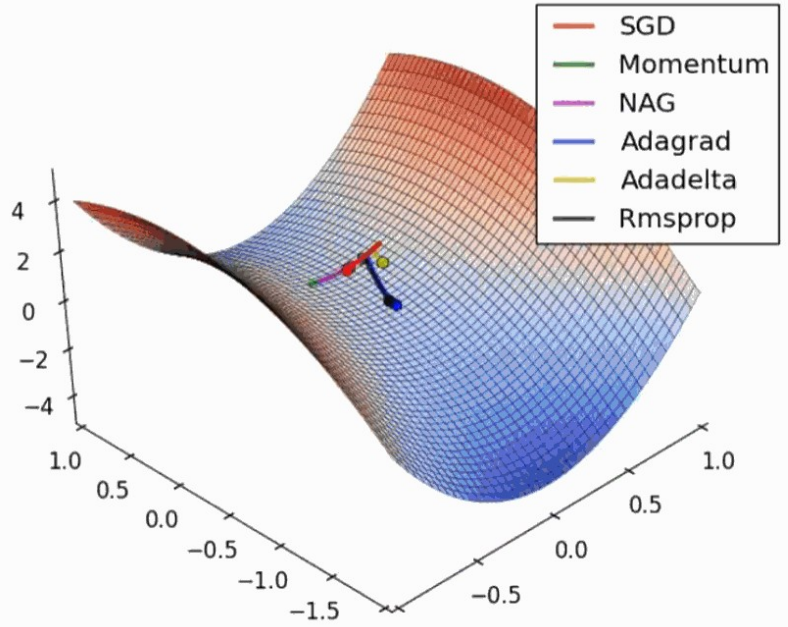
1. Sample m (batch size) samples $\{x^{(1)}, \ldots, x^{(m)}\}$ from the training s set with corresponding targets $\{y^{(s)}\}$
2. Compute gradient: $g \leftarrow \frac{1}{m} \nabla_\theta \sum_i L\left(f(xx^i); \theta\right), y^i)$
3. Accumulate gradient: $v \leftarrow \mu v + (1 - \mu) g$
4. Accumulate sqadient: $\tilde{r} \leftarrow \rho \tilde{r} + (1 - \rho) g * g$
5. Correct bias: $\tilde{v} = \frac{v}{v}(1 - \mu^t); \tilde{r} = \frac{v}{r}(1 - \rho^t)$
6. Update weights: $\theta \leftarrow \theta - \epsilon / \left(\sqrt{\tilde{r}} + \delta\right) * \tilde{v}$

end while

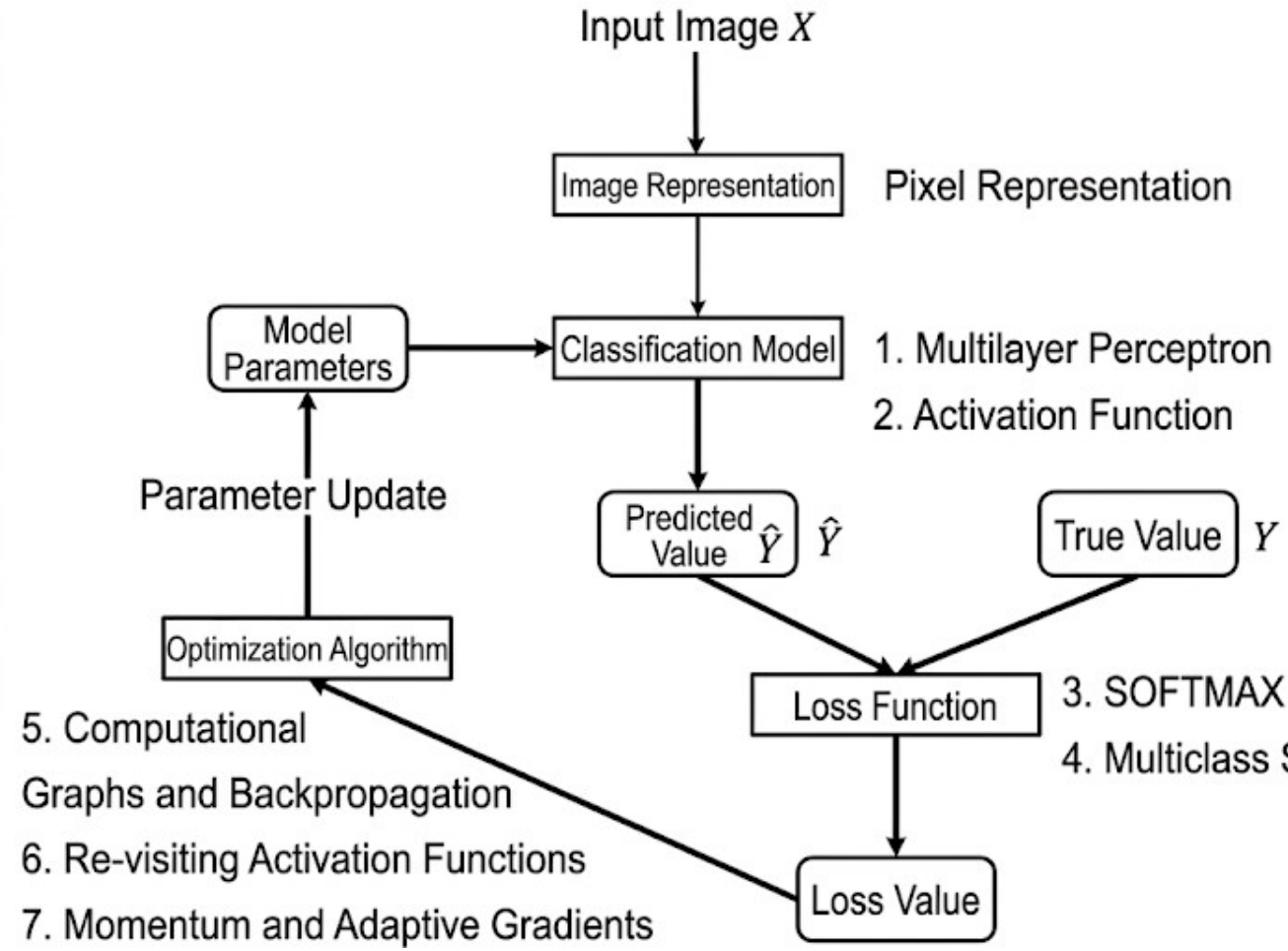
- Simultaneously uses momentum and adaptive gradient ideas
- The bias correction step can greatly alleviate the cold start problem in the early stage of the algorithm

Summary





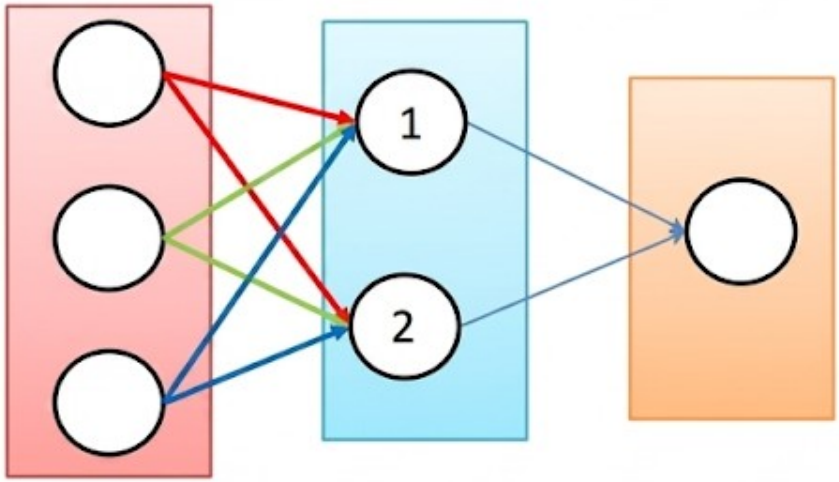
Saddle Points and SGD



Training Process

8. Weight Initialization
9. Batch Normalization
10. Underfitting, Overfitting, and Dropout
11. Model Regularization and Hyperparameter Tuning

Weight Initialization



Suggestion: Adopt random initialization, avoid all-zero initialization!

All-zero Initialization: Different neurons in the network will have the same output and perform the same parameter updates; Therefore, the parameters learned by these neurons will all be the same, which is equivalent to a single neuron.

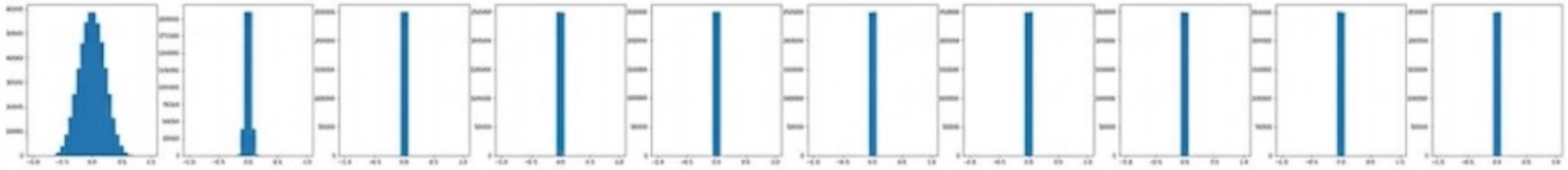
Random Weight Initialization

Network Structure: 10 hidden layers, 1 output layer, each hidden layer containing 500 neurons, using the hyperbolic tangent activation function.

Random Initialization:

Weights are $N(0,0.01)$ from a Gaussian distribution

```
input layer had mean 0.000097 and std 0.999691
hidden layer 1 mean -0.000134 and std 0.214021
hidden layer 2 mean 0.000025 and std 0.047655
hidden layer 3 mean -0.000030 and std 0.010683
hidden layer 4 mean -0.000002 and std 0.002390
hidden layer 5 mean -0.000000 and std 0.000532
hidden layer 6 mean -0.000000 and std 0.000119
hidden layer 7 mean -0.000000 and std 0.000027
hidden layer 8 mean 0.000000 and std 0.000006
hidden layer 9 mean 0.000000 and std 0.000001
hidden layer 10 mean 0.000000 and std 0.000000
```



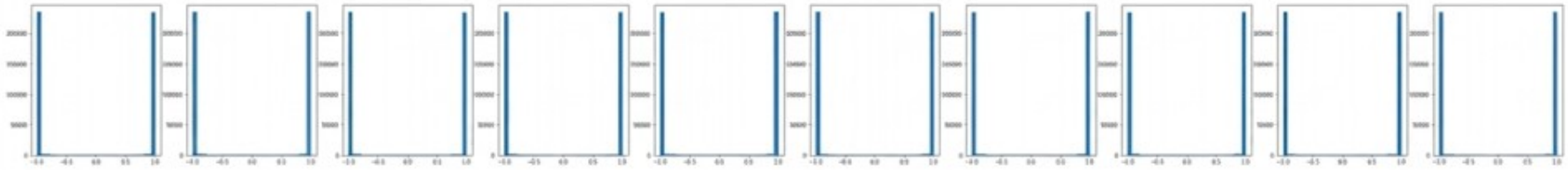
Except for the first two layers, the activation values of all subsequent layers are 0; in this case, the input information cannot pass to the output layer; eventually, the network will not be trained.

RANDOM WEIGHT INITIALIZATION

Network structure: 10 hidden layers, 1 output layer, each hidden layer contains 500 neurons, using the hyperbolic tangent activation function.

Random initialization: Weights are sampled from a sampled ra $N(0,1)$ Gaussian distribution.

```
input layer had mean 0.000240 and std 1.000121
hidden layer 1 mean -0.003320 and std 0.982045
hidden layer 2 mean 0.002391 and std 0.981602
hidden layer 3 mean 0.000433 and std 0.981700
hidden layer 4 mean 0.000040 and std 0.981740
hidden layer 5 mean 0.000358 and std 0.981791
hidden layer 6 mean -0.000324 and std 0.981612
hidden layer 7 mean 0.002807 and std 0.981484
hidden layer 8 mean -0.001022 and std 0.981578
hidden layer 9 mean 0.000497 and std 0.981684
hidden layer 10 mean -0.001396 and std 0.981915
```



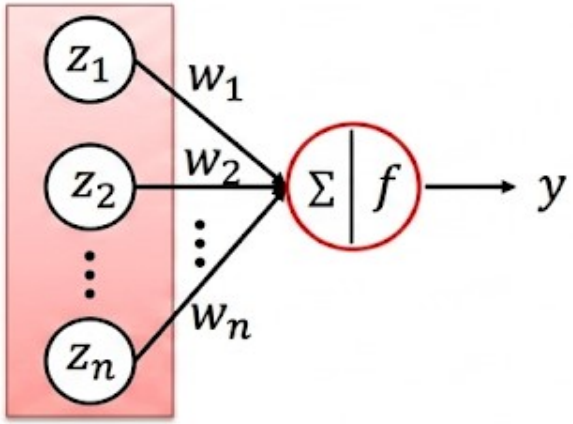
Almost all neurons are saturated (either -1 or 1); at this time, the local gradients of the neurons are zero, and the network has no backpropagation gradient flow; ultimately, all parameters are not updated.

Random weight initialization

- Experimental conclusion: Making the weights unequal during initialization does not ensure that the network can be trained properly.
- Effective initialization method: Keep the variance of activation values and local gradients in each layer of the network as consistent as possible during propagation, so as to maintain the forward and backward data flow in the network.

XAVIER INITIALIZATION

Consider a single neuron with inputs $z_1, z_2, \ldots, z_n$, which are independent and identically distributed. Its weights are $w_1, \ldots, \ldots, w_n$, which are also independent and Identically distributed, and *w* and *z* are independent. The activation function f. The expression for the final output *y* is:



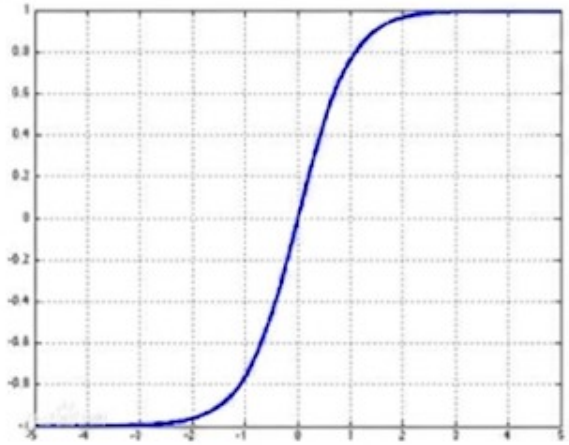
$$y = f(w_1 * z_1 + \cdots + w_n * z_n)$$

Goal: Make the variance of the activation values and local gradients consistent across all layers of the network during propagation; i.e., find the distribution of *w* such that the variance of the output *y* is consistent with that of the input *z*.

Xavier Initialization

Consider a single neuron with inputs $z_1, z_2, \cdots, z_N$, where these N inpats are independent and identically distributed (i.i.d.). Its weights $w_1, \ldots, w_N$ are also i.i.d., and w and z are independent. The activation functiion is f. The expression for the final output y is:

$$y = f(w_1 * z_1 + \cdots + w_N * z_N)$$



Assume f is the hyperbolic tangent function. $w_1, \ldots, w_N$ are i.i.d. random variables. $z_1, \ldots, z_N$ are i.i.d. random variables, and w is independent of z. All have a mean of 0. Then, we have:

$$\begin{aligned} Var(y) &= Var\left(\sum_{i=1}^{n} w_i z_i\right) = \sum_{i=1}^{n} Var(w_i z_i) \\ &= \textstyle\sum_i^n [E(w_i)]^2 Var(z_i) + [E(z_i)]^2 Var(w_i) + Var(w_i)Var(z_i) \\ &= \textstyle\sum_i^n Var(w_i)Var(z_i) \\ &= nVar(w_i)Var(z_i) \end{aligned}$$

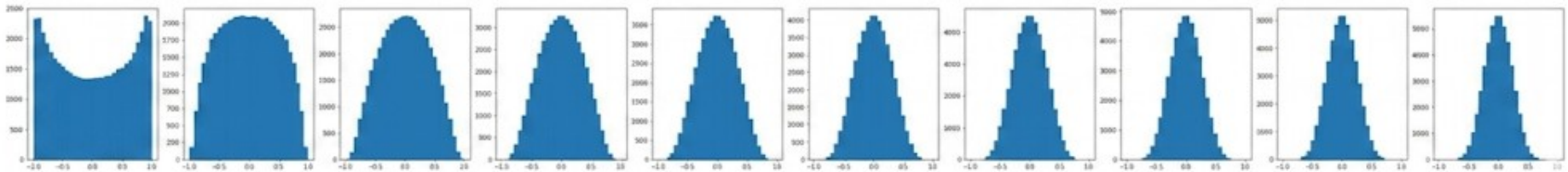
When $\text{var}(w) = 1/N$, the variance of y is consistent with the variance of z.

Xavier Initialization

Network structure: 10 hidden layers, 1 output layer, each hidden layer containing 500 neurons, using the hyperbolic tangent activation function.

Random initialization: Weights are sampled from om a $\mathcal{N}(0,1/N)$ Gaussian distribution, where N is the number of input neurons.

Achievement of goal: The variance of the neuron activation values of each layer is basic the same!



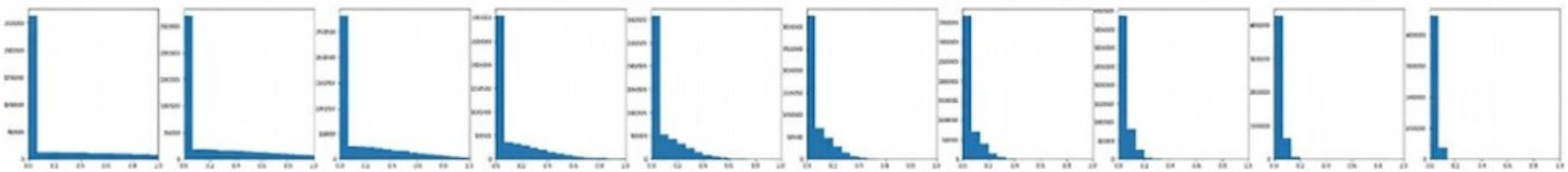
Xavier Initialization

Network structure: 10 hidden layers, 1 output layer, each hidden layer containing 500 neurons, using the ReLU activation function.

Random initialization:

Weights are sampled from a $N(0,1/N)$ Gaussian distribution, where N is the number of input neurons.

The Xavier initialization method is not quite suitable for the ReLU activation function!



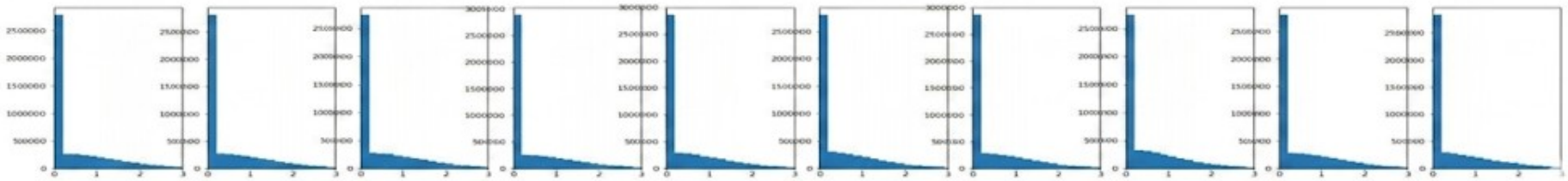
HE INITIALIZATION (MSRA)

Netwsture: 10 hidden layers, 1 outulayer, each hidden layer contains 500 neurons, using ReLU activation.

Random initialization: Weights are sampled from $N(0,2/N)$ Gaussian distribution, N is the number of input neurons.

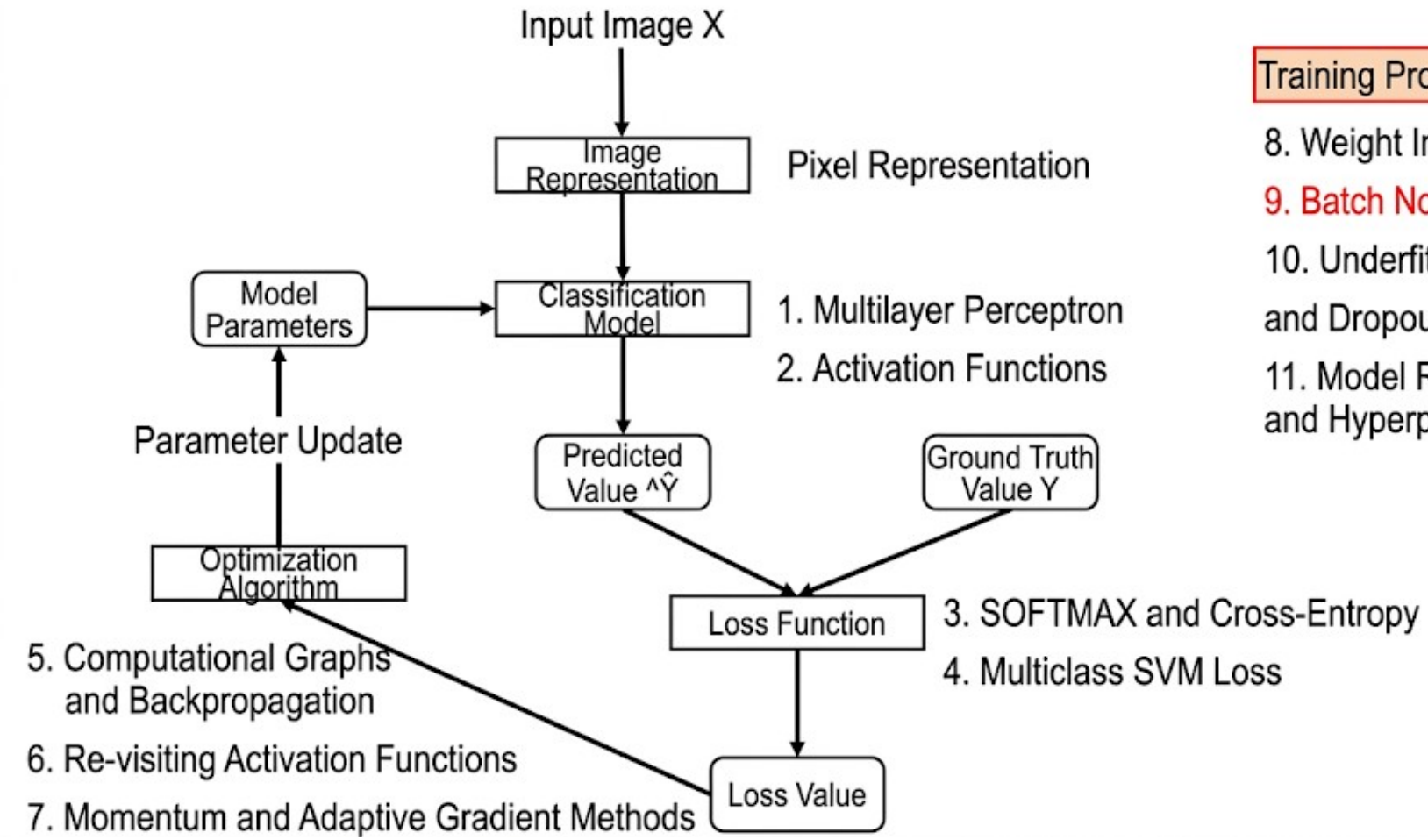
ReLU Activation Function Initialization Method

Modified Here



Summary of Weight Initialization

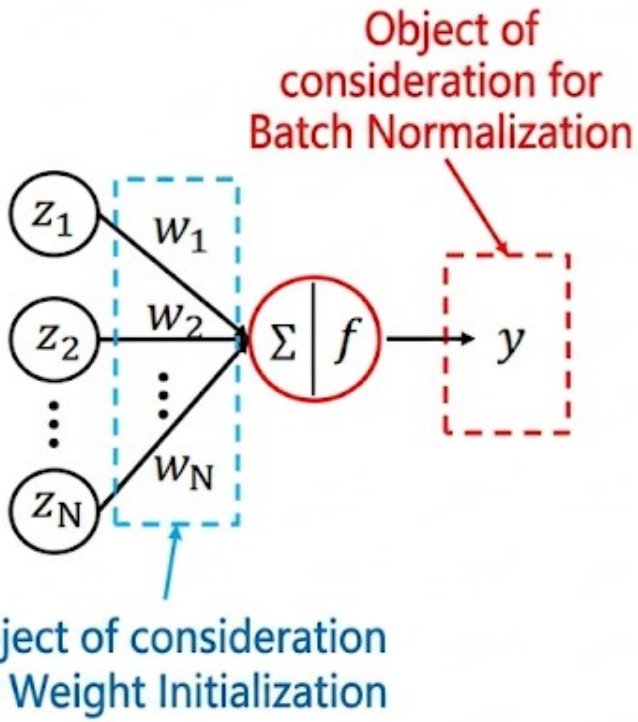
- A good initialization method can prevent information loss during forward propagation and also solve the vanishing gradient problem during backpropagation.
- When the activation function is tanh or Sigmoid, the Xavier initialization method is recommended.
- When the activation function is ReLU or Leaky ReLU, the He initialization method is recommended.



Training Process

8. Weight Initialization
9. Batch Normalization
10. Underfitting, Overfitting, and Dropout
11. Model Regularization and Hyperparameter Tuning

Batch Normalization



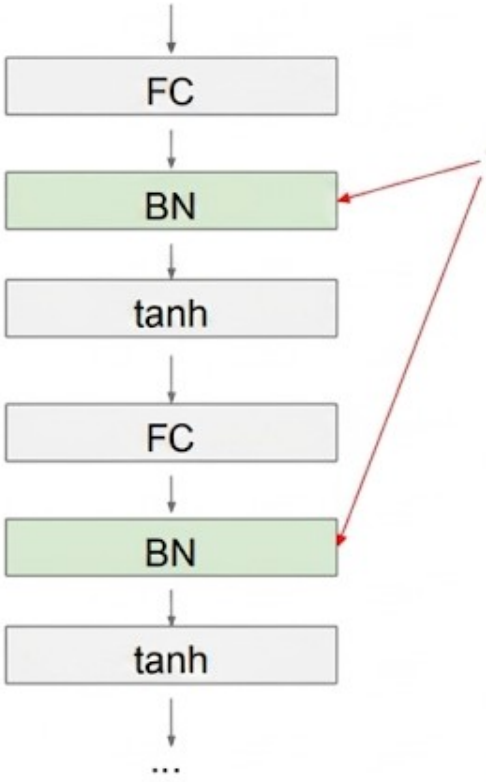
Method: Adjust the weight distribution so that input and output have the same distribution.

Method: Directly normalize the outputs of the neurons

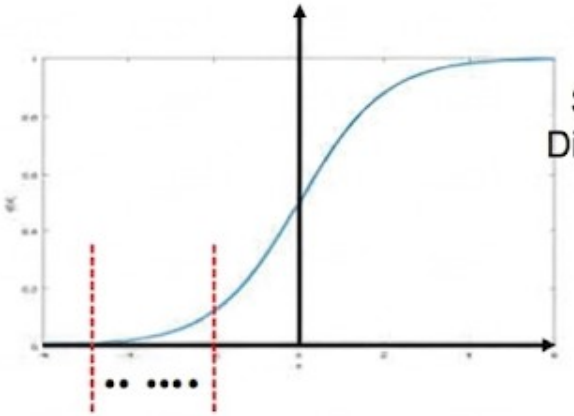
- Recall of Min-batch Gradient Descent: Each iteration reads a batch of data, e.g., 32 samples; after passing through the current neuron, there will be $y_1, \dots, y_{32}$.
- Batch Normalization operation: For these 32 outputs, perform the operation of mean subtraction and division by standard deviation (or variance); this can guarantee that the distribution of the current neuron's output values fits a distribution with a mean of 0 and a variance of 1.

If each neuron in each layer is normalized, the signal vanishing problem in the forward propagation process can be solved.

Batch Normalization and Vanishing Gradient

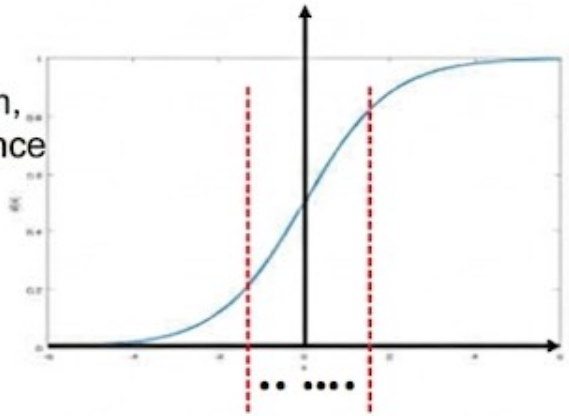


Often inserted after Fully-Connected layer,
before non-linear activation



Low gradient,
zero-gradient region

Subtract mean,
Divide by variance



Batch Normalization

Batch Normalization Algorithm

Input: $B = \{x_1, \cdots, x_m\}$

Learning parameters: γ, β

Output: $\{y_1, \cdots, y_m\}$

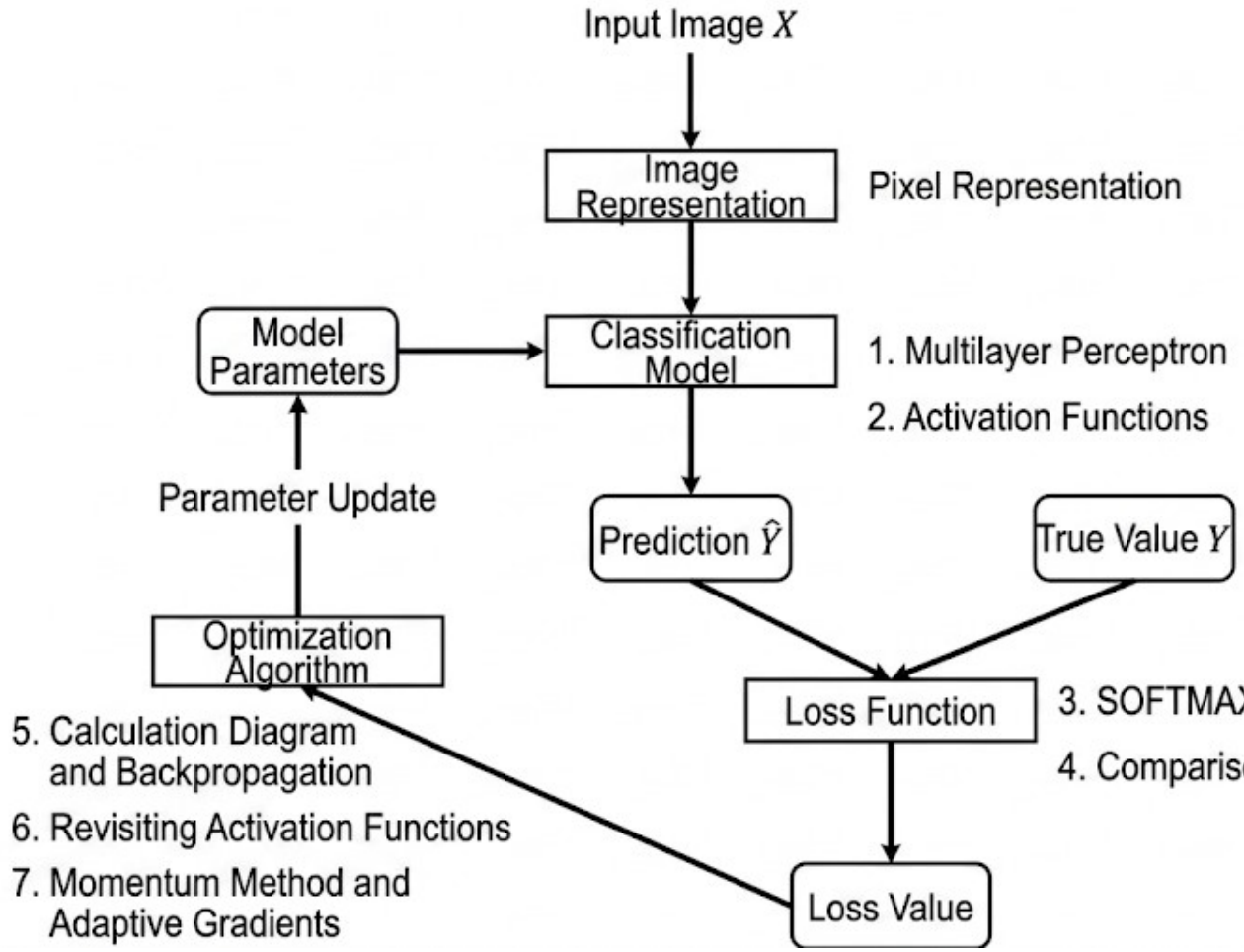
1. Calculate mini-batch mean: $\mu_B \leftarrow \frac{1}{m}\sum x_i$
2. Calculate mini-batch:: $\sigma_B^2 \leftarrow \frac{1}{m}\sum (x_i - \mu_B)^2$
3. Normalize: $\hat{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$
4. Shift and Scale: $y_i \leftarrow \gamma \hat{x}_i + \beta$

Q: Is the output standard normal distribution with 0 mean and 1 variance the best distribution for network classification?

A: It is better to determine the data distribution's mean and variance based on their contribution to classification.

Q: For single-sample inference, how are mean and variance set?

A: **From training. Accumulate the mean and variance of each batch during training, then average them. Use the average thickness result** as (running statistics) as the mean and variance for prediction.



Training Process

8. Weight Initialization
9. Batch Normalization
10. Underfitting, Overfitting and Dropout
11. Model Regularization and Hyperparameter Tuning

Overfitting Phenomenon

When overfitting occurs, the resulting model achieves high accuracy on the training set, but its recognition rate is very low in real-world scenarios.

Overfitting and Underfitting

Overfitting — occurs when the model selected during learning contains too many parameters, …
to the extent that this model predicts seen data very well, but predicts unseen data poorly. In this situation,
the model may have merely memorized the training set data, instead of having learned the data features.

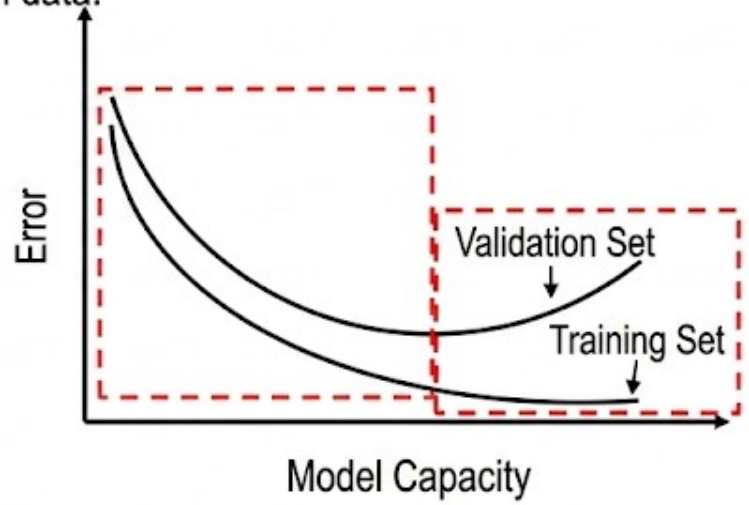
Underfitting — the model description ability is too weak, to the extent that it cannot learn the patterns in the data well.

The reason for underfitting is usually that the model is too simple.

Overfitting in the Learning Process

- The fundamental problem of machine learning is optimization and generalization.
- Optimization — refers to adjusting the model to achieve optimal performance on training data;
- Generalization — refers to the quality of performance of a trained model on previously unseen data.

Early training phase: Optimization and generalization are related: are related: The smaller the error on the training set, the smaller the error on the validation set, and the model's generalization ability gradually increases.



Late training phase: The error rate on the validation set no longer decreases, but instead starts to increase. Overfitting occurs in the model, as it begins to learn patterns related *only* to the training data.

Addressing Overfitting

- Optimal Solution —— Acquire more training data
- Second-best Solution —— Adjust the amount of information the model is allowed to store, or constrain the information the model is allowed to store. These types of methods are also called regularization.
  - Adjust model size
  - Constrain model weights, i.e., weight regularization (Common ones include L1, L2 regularization)

L2 Regularization

$$L(W) = \underbrace{\frac{1}{N}\sum_i L_i\left(f(x_i, W), y_i\right)}_{\text{Data Loss}} + \underbrace{\lambda R(W)}_{\text{Weight Regularization Loss}}$$

L2 Regularization Loss: $R(W) = \sum_k \sum_l W_{k,l}^2$

L2 Regularization Loss penalizes large weight vectors heavily, encouraging more diffuse weight vectors,
making the model tend towards using all input features for decision making.
The model generalization is good at this time!

Addressing Overfitting

Optimal Solution —— Obtain more training data

Next Optimal Solution —— Adjust the model's capacity to store information or apply constraints to the model's storage, these types of methods are also called regularization.

- Adjust Model Size
- Constrain Model Weights, i.e., Weight Regularization (Common ones include L1, L2 regularization)
- **Dropout**

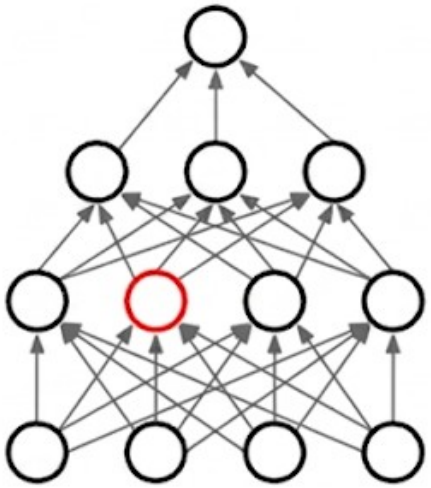
Dropout

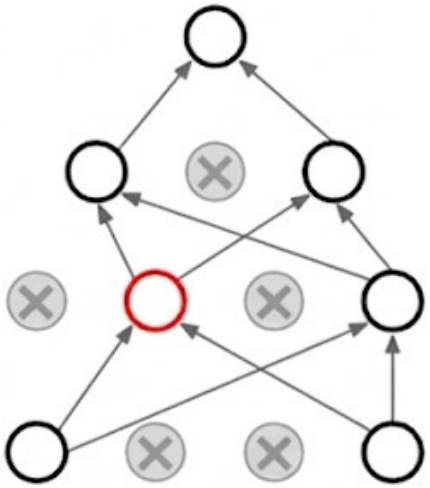
- Dropout: Makes a fraction of neurons in hidden layers inactivate randomly.
- Implementation: Using Dropout in a certain layer during training means randomly discarding some outputs from this layer (setting their values to 0). These discarded neurons act as if they have been deleted from the network.
- Dropout ratio: The proportion of features set to 0, usually in the range of 0.2—0.5.

Example: Suppose a certain layer's return value for a given input sample should be the vector: [0.2, 0.5, 1.3, 0.8, 1.1].
After using Dropout, this vector will have a few random elements changed to 0:
[0, 0.5, 1.3, 0, 1.1].

Dropout (Random Deactivation)

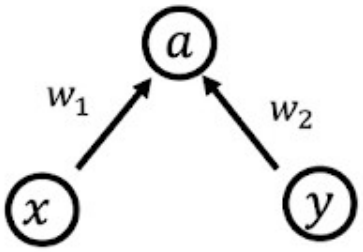
- Why can random deactivation prevent overfitting?
- Explanation 1: Random deactivation reduces the network parameters involved in computation each time the gradient is updated, lowering the model capacity, so it can prevent overfitting.
- Explanation 2: Random deactivation encourages weight diffusion, so from this perspective, random deactivation also acts as regularization, further preventing overfitting.
- Explanation 3: Dropout can be viewed as model ensemble





Applications of Dropout

The existing problem:



Testing phase: $E[a] = w_1 x + w_2 y$

Training phase: $E[a] = \frac{1}{4}(w_1 x + w_2 y) + \frac{1}{4}(w_1 x + 0y)$
$+ \frac{1}{4}(0x + w_2 y) + \frac{1}{4}(0x + 0y)$
$= \frac{1}{2}(w_1 x + w_2 y)$

Application of Dropout

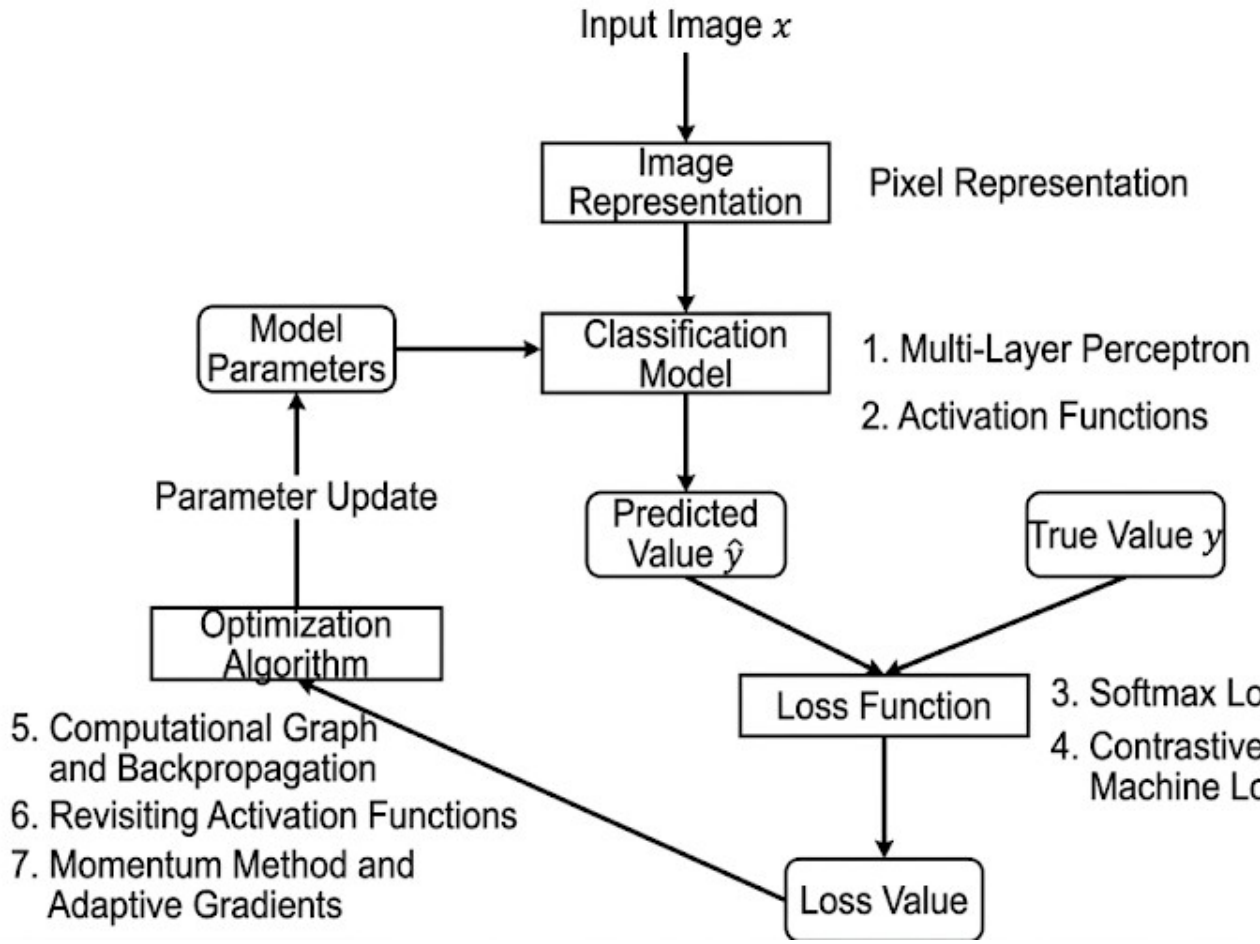
Example of a Three-Layer Neural Network:

```
p = 0.5 # Probability of neurons remaining in the active st
def train(X):
    H1 = np.maximum(0, np.dot(W1,X) + b1)
    U1 = np.random.rand(*H1.shape) < p # First layer mask
    H1 *= U1 # First layer dropout operation
    H2 = np.maximun(0, np.dot(W2,H1) + b2)
    U2 = np.random.rand(*H2.shape) < p # Second layer mask
    H2 *= U2 # Second layer dropout operation
    out = np.dot(W3,H2) + b3

def predict(X):
    H1 = np.maximum(0, np.dot(W1,X) + b1) * p
    H2 = np.maximun(0, np.dot(W2,H1) + b2) * p
    out = np.dot(W3,H2) + b3
```

Application of Dropout

Example of a Three-Layer Neural Network:

```
p = 0.5 # Probability of neurons remaining in the active state; the higher the value, the fewer deactivated units
def train_new(X):
    H1 = np.maximum(0, np.dot(W1,X) + b1)
    U1 = (np.random.rand(*H1.shape) < p) / p # First layer mask, note the division by p
    H1 *= U1
    H2 = np.maximun(0, np.dot(W2,H1) + b2)
    U2 = (np.random.rand(*H2.shape) < p) / p # Second layer mask, note the division by p
    H2 *= U2
    out = np.dot(W3,H2) + b3

def predict_new(X):
    H1 = np.maximum(0, np.dot(W1,X) + b1)
    H2 = np.maximun(0, np.dot(W2,H1) + b2)
    out = np.dot(W3,H2) + b3
```



Training Process

8. Weight Initialization
9. Batch Normalization
10. Underfitting, Overfitting, and Dropout
11. Model Regularization and Hyperparameter Tuning

Hyperparameters in Neural Networks

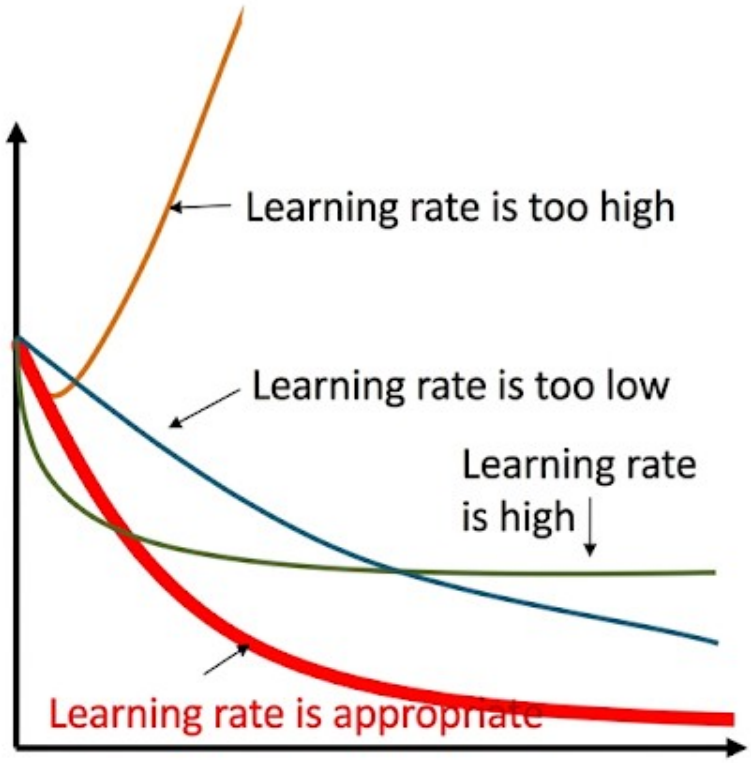
Hyperparameters:

- Network Structure—Number of hidden layer neurons, number of network layers, selection of non-linear units, etc.

- Optimization Related—Learning rate, dropout ratio, regularization term strength, etc.

Importance of Hyperparameters,
how to find suitable Hyperparameters?

Learning Rate Setting



- Learning rate is too large, the training process fails to converge.
- Learning rate is biased towards being large, oscilating near the minimum value, reach optimal.
- Learning rate is too small, convergence time is relatively long.
- Learning rate is appropriate, rapid convergence, good results.

Hyperparameter Optimization Methods

Grid Search Method:

① For each hyperparameter, take several values respectively, combine these hyperparameter values to form multiple sets of hyperparameters;

② Evaluate the model performance for each set of hyperparameters on the validation set;

③ Select the set of values used by the model with the best performance as the final hyperparameter values.

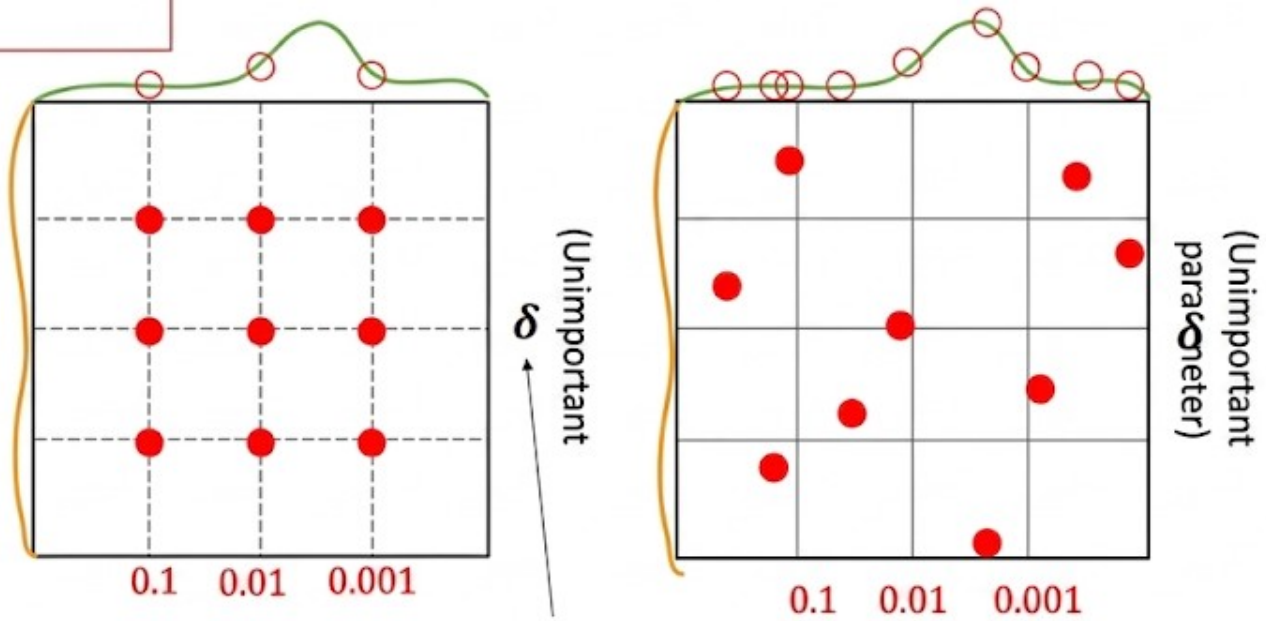
Hyperparameter Optimization Methods

Budget: 9 experiments

Goal: best hyperparameter combination

Grid Search

Random Search



Tried three different learning rates

Tried nine different learning rates!

δ (Unimportant parameter)

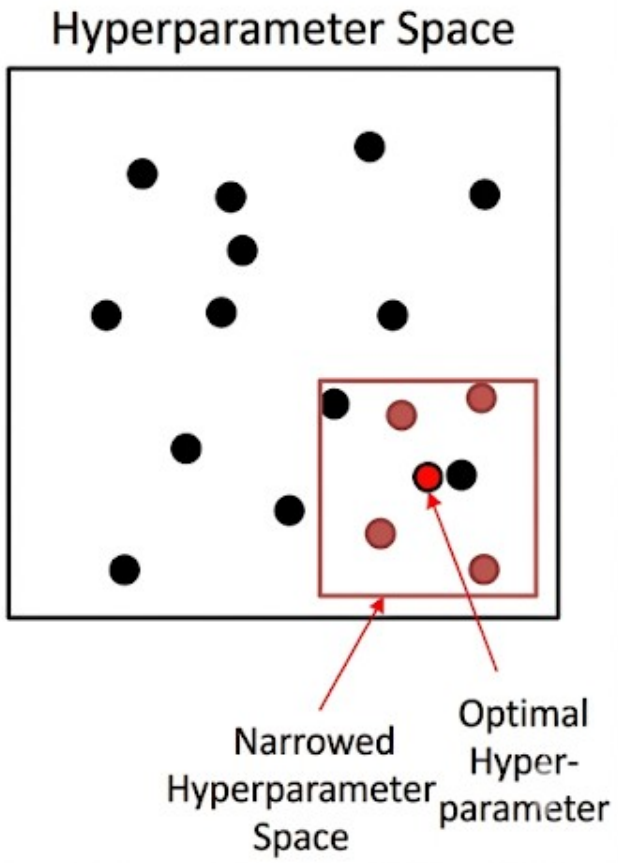
Learning Rate (Important parameter)

Prevent division-by-zero parameter, $\delta(10^{-5}, 10^{-6}, 10^{-7})$

Hyperparameter Search Strategy

Hyperparameter Search Strategy:

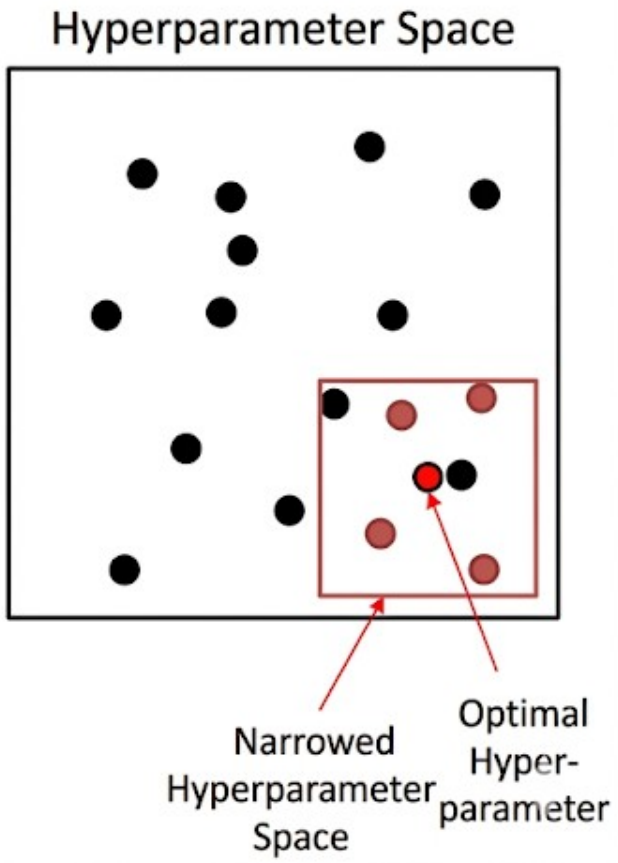
- Coarse Search: Sample hyperparameters over a wide range using random methods. Train for one epoch, and narrow the hyperparameter range based validation set accuracy.



Hyperparameter Search Strategy

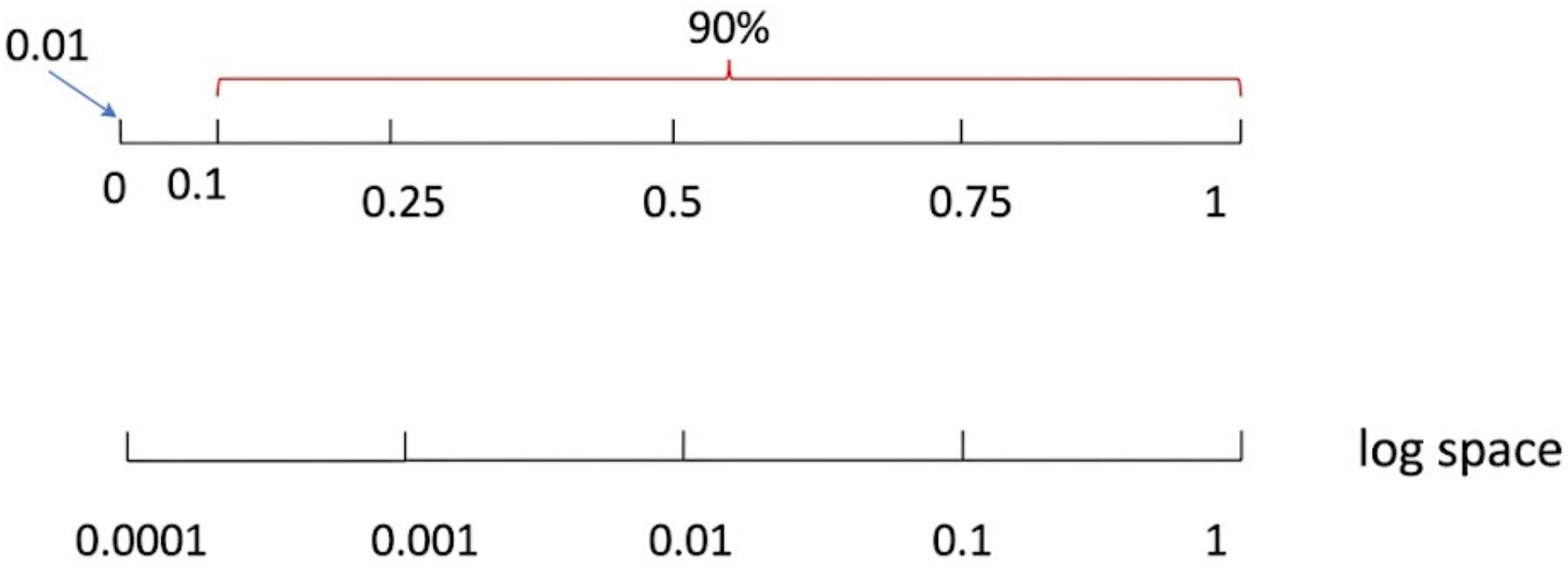
Hyperparameter Search Strategy:

- Coarse Search: Sample hyperparameters over a wide range using random methods. Train for one epoch, and narrow the hyperparameter range based validation set accuracy.
- Fine Search: Sample hyperparameters within the aforementioned narrowed range using random methods. Run the model for fiveto ten epochs, and select the set of hyperparameters with the highest accuracy on the validation set.

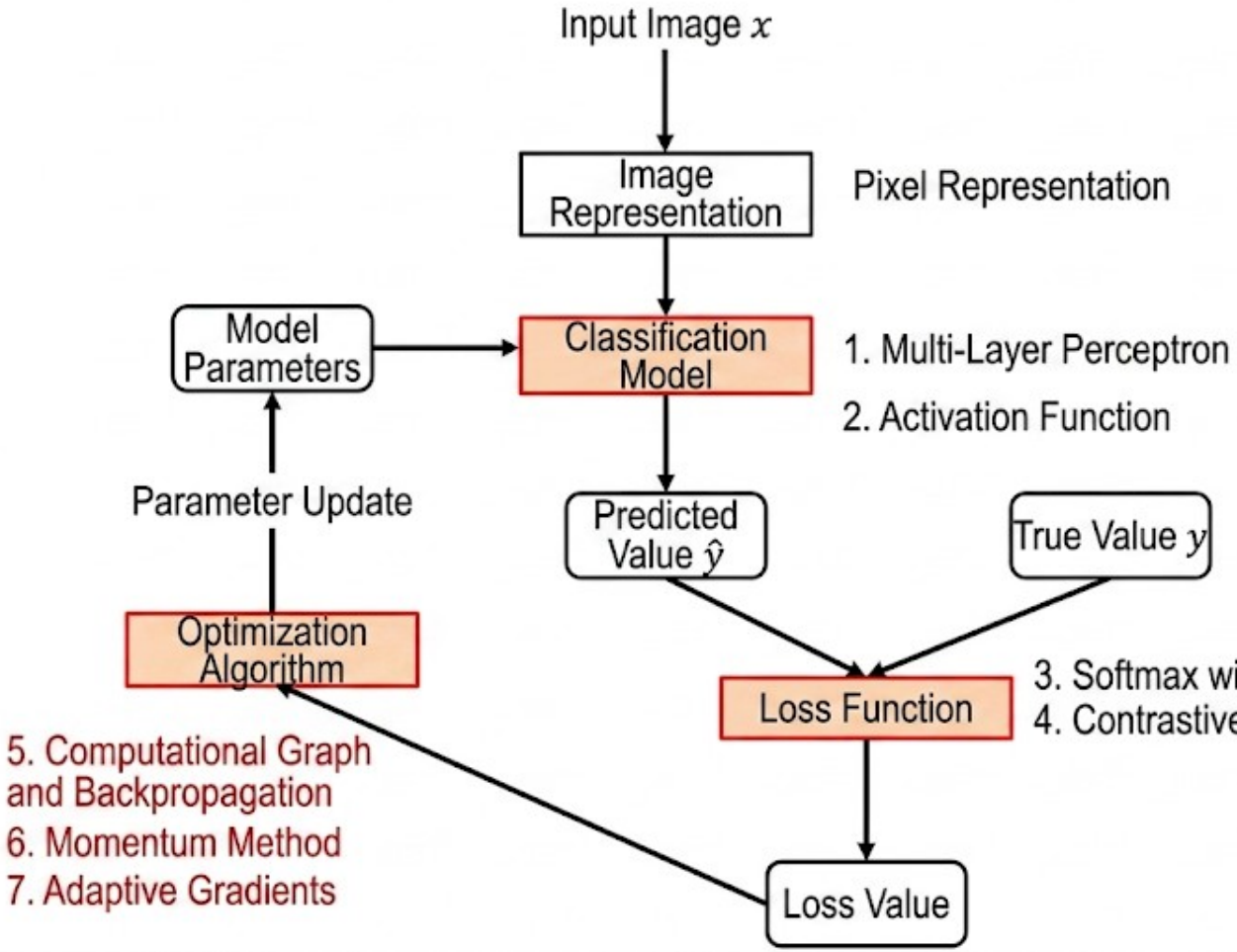


Hyperparameter Scale Space

Taking the learning rate as an example:



Recommendation: For hyperparameters such as learning rate and regularization term strength, random sampling in log space is more suitable!



Training Process

8. Weight Initialization
9. Batch Normalization
10. Underfitting and Overfitting
11. Model Regularization and Hyperparameter Tuning